

Teaching masked diffusion language models to generate chess puzzles with reinforcement learning

Aatu Selkee

School of Science

Thesis submitted for examination for the degree of Master of
Science in Technology.

Espoo July 3, 2026

Supervisor

Prof. Nuutti Hyvönen

Advisor

Adjunct Prof. Eric Malmi

Copyright © 2026 Aatu Selkee

The document can be stored and made available to the public on the open internet pages of Aalto University.
All other rights are reserved.

Author Aatu Selkee

Title Teaching masked diffusion language models to generate chess puzzles with reinforcement learning

Degree programme Masters's Programme in Mathematics and Operations Research

Major Applied Mathematics

Code of major SCI3053

Supervisor Prof. Nuutti Hyvönen

Advisor Adjunct Prof. Eric Malmi

Date July 3, 2026

Number of pages 55+7

Language English

Abstract

Traditionally, creativity has been considered a fundamental measure of machine intelligence. While the recent advancement of large language models (LLMs) has enabled a wide array of new, creative applications, generating outputs that demand both creativity and reasoning remains a significant challenge. This thesis addresses this limitation by utilizing generative models to create chess puzzles. Chess puzzles are inherently delicate, as altering even a single piece can completely invalidate the composition. Consequently, chess puzzle generation serves as an excellent playground to benchmark and enhance the reasoning and creative capabilities of language models.

While autoregressive LLMs currently dominate language modelling, discrete diffusion models have recently emerged as a compelling alternative. In particular, masked diffusion models, which generate sequences through iterative unmasking, offer unique advantages. Notably, they grant the user manual control over inference speed while maintaining text quality comparable to traditional LLMs. They, however, require more pretraining to achieve the same performance. In this work, we employ masked diffusion models to generate creative and counter-intuitive chess puzzles conditioned on specific themes, granting users fine-grained control over the generated puzzle types.

Our methodology begins with pretraining on the Lichess puzzle dataset. Because valid chess puzzles must satisfy strict criteria, most importantly, possessing a unique best solution, we subsequently apply reinforcement learning (RL) to optimize the model directly for these constraints. We utilize Denoising Diffusion Policy Optimization (DDPO) for the optimization process. Our training specifically optimizes for solution uniqueness, counter-intuitiveness, theme accuracy, and creativity, among other metrics.

Our results demonstrate the effectiveness of this approach. While supervised training enables the model to generate valid puzzles 0.18% of the time, the application of reinforcement learning doubles this success rate to 0.37%. Furthermore, we show that incorporating the auxiliary tasks of simultaneously generating the best move alongside the puzzle, further enhances puzzle quality. Ultimately, this thesis highlights the potential of masked diffusion models combined with reinforcement learning for solving complex generation tasks.

Keywords Masked diffusion, dLLMs, Reinforcement learning, DDPO, Chess puzzles



Tekijä Aatu Selkee

Työn nimi Vahvistusoppimisen käyttäminen shakkitehtävien luomiseen
naamiodiffuusiomalleilla

Koulutusohjelma Matematiikan ja operaatiotutkimuksen maisteriohjelma

Pääaine Soveltava matematiikka

Pääaineen koodi SCI3053

Työn valvoja Prof. Nuutti Hyvönen

Työn ohjaaja Adjunct Prof. Eric Malmi

Päivämäärä July 3, 2026

Sivumäärä 55+7

Kieli Englanti

Tiivistelmä

Perinteisesti luovuutta on pidetty koneälykkyyden tärkeimpänä mittarina. Vaikka suurten kielimallien (LLM) viimeaikainen kehitys on mahdollistanut uusia sovelluksia, on luovuutta ja päättelykykyä vaativien vastauksien luominen edelleen haastavaa. Tämä tutkielma käsittelee tätä rajoitetta käyttämällä generatiivisia malleja shakkitehtävien luomiseen. Shakkitehtävät ovat luonnostaan herkkiä, sillä yhdenkin nappulan vaihtaminen voi hajottaa tehtävän. Tästä syystä shakkitehtävien luominen on erinomainen ongelma kielimallien kykyjen testaamiseen ja parantamiseen.

Vaikka autoregressiiviset kielimallit hallitsevat kielen mallinnusta tällä hetkellä, diskreetit diffuusiomallit ovat kehittyneet varteenotettaviksi vaihtoehdoiksi viime vuosina. Erityisesti naamiodiffuusiomallit, jotka tuottavat tekstiä luomalla toistuvasti muutamia tavuja kerrallaan, tarjoavat joitakin etuja autoregressiivisiin kielimalleihin nähden. Ne muun muassa antavat käyttäjälle mahdollisuuden vaikuttaa tekstin luomisnopeuteen manuaalisesti ylläpitäen vastaavaa tekstin laatua kuin autoregressiivisilläkin malleilla. Niiden kouluttaminen kuitenkin vaatii enemmän laskentaa saman suorituskyvyn saavuttamiseksi. Tässä tutkielmassa käytämme naamiodiffuusiomalleja luomaan tiettyihin teemoihin ehdollistettuja, luovia ja epäintuitiivisia shakkitehtäviä.

Tutkimus alkaa mallin esikoulutuksella, jossa käytetään Lichess:n shakkipulma tietoaaineistoa. Koska hyvien shakkipulmien tulee täyttää tietyt kriteerit, käytetään vahvistusoppimista pulmien ominaisuuksien parantamiseen. Mallissa parannetaan erityisesti sen kykyä luoda pulmia, joilla on yksikäsitteinen ratkaisu. Siihen käytetään Denoising Diffusion Policy Optimization (DDPO) algoritmia. Algoritmi optimoi muun muassa ratkaisun yksikäsitteisyyttä, epäintuitiivisuutta, teeman tarkkuutta, sekä luovuutta.

Tulokset osoittavat menetelmän toimivuuden. Esikoulutusvaiheen jälkeen, malli loi kriteerit täyttäviä pulmia noin 0.18% ajasta. Vahvistusoppimisella pulmien osuus tuplaantui 0.37%:iin. Tulokset osoittavat myös, että parhaan siirron ennustaminen pulmien luomisen lisäksi parantaa pulmien ominaisuuksia. Pohjimmiltaan tämä työ korostaa naamiodiffuusiomallien ja vahvistusoppimisen potentiaalia kielen mallinnuksessa.

Avainsanat Naamiodiffuusio, dLLM:t, vahvistusoppiminen, DDPO, shakkipulmat

Preface

I would like to express my sincere gratitude to my advisor, Adjunct Professor Eric Malmi, whose mentorship and ideas made this work possible. I am equally grateful for the privilege of working within his research group. Furthermore, I am also deeply grateful to Professor Nuutti Hyvönen for his advice and support throughout the writing process. Additionally, I would like to thank Dr. Severi Rissanen on his guidance on the theory of diffusion models and the suggestion to utilize DDPO, as well as Dr. Tom Zahavy and Dr. Xidong Feng for showcasing their work and giving feedback on our research. Finally, I would like to thank my family and friends whose encouragement kept me going even when progress was slow.

Otaniemi, 3.7.2026

Aatu Selkee

Contents

Abstract	3
Abstract (in Finnish)	4
Preface	5
Contents	6
Symbols and abbreviations	8
1 Introduction	10
2 Literature review	12
3 Background	14
3.1 Discrete Diffusion Models	14
3.1.1 Forward process	14
3.1.2 Reverse process	16
3.1.3 Evidence lower bound	16
3.1.4 Sampling	17
3.2 Reinforcement Learning	19
3.2.1 Policy gradient theorem	19
3.2.2 Proximal policy optimization	20
3.2.3 Denoising diffusion policy optimization	21
3.2.4 Kullback-Leibler divergence and entropy	22
3.3 Chess terminology	26
4 Methodology	28
4.1 Tokenization	28
4.2 Defining chess puzzles	29
4.2.1 Legality	29
4.2.2 Uniqueness	29
4.2.3 Counter-intuitiveness	30
4.2.4 Reward function	31
4.2.5 Model evaluation criteria	34
4.3 Supervised training	34
4.4 Reinforcement learning algorithm	36
5 Results	38
5.1 Supervised results	38
5.2 Reinforcement learning results	41
5.3 Qualitative results	44
6 Conclusion	45

A Theme computation	56
B Supervised training	57
C Reinforcement learning	59
C.1 Additional results	59
C.2 ELBO-based Sequence-level Policy Optimization	61
D Implementation details	62

Symbols and abbreviations

Symbols

e_i	Basis vector where the i -th element is one and the rest zeros.
$\text{Cat}(x; p)$	Probability mass function of a categorical distribution.
α_t	Masking schedule of a diffusion model.
μ_θ	Denoising network of a diffusion model.
p_{unmask}	Probability of a masked token being unmasked.
$\mathcal{L}_\theta(y x)$	Evidence lower bound.
$\mathcal{M}$	Markov decision process.
$\mathcal{S}$	Set of states in an MDP.
$\mathcal{A}$	Set of actions in an MDP.
P	Transition function in an MDP.
$\Delta(\mathcal{S})$	Set of probability distributions over a set $\mathcal{S}$.
$\pi(s, a)$	Policy (decision-making rule in an MDP).
G_t	Cumulative reward on step t .
$V(s)$	State-value function.
$Q(s, a)$	Action-value function.
$J(\theta)$	Expected episodic reward.
$\hat{A}_t$	Advantage function.
$H(p)$	Entropy of a distribution p .
$H_b(\cdot)$	Binary entropy function.
$\mathbb{KL}(\pi_\theta \parallel \pi_{\text{ref}} s_i)$	KL divergence between the current and reference policies.
$pv(s)$	Principal variation of a position s .
$w(a s)$	Winning chances after a move a in position s in a chess game.
$\mathbb{I}_{\text{legal}}(s)$	Legality criterion.
$\mathbb{I}_{\text{unique}}(s)$	Uniqueness criterion.
$\mathbb{I}_{\text{counter-intuitive}}(s)$	Counter-intuitiveness criterion.
$r_{\text{counter-intuitive}}(s)$	Counter-intuitiveness value function.
$\mathbb{I}_{\text{filter}}(s)$	Diversity, realism and theme filter.
$R(s)$	Reward function.

Abbreviations

AI	Artificial Intelligence
CDF	Cumulative distribution function
DDPO	Denoising Diffusion Policy Optimization
dLLM	Diffusion Large Language Model
ELBO	Evidence Lower Bound
ESPO	ELBO-based Sequence-level Policy Optimization
FEN	Forsyth-Edwards Notation
GPU	Graphics Processing Unit
GSPO	Group Sequence Policy Optimization
KL	Kullback-Leibler
LLM	Large Language Model
MC	Monte-Carlo
MDP	Markov Decision Process
NLP	Natural Language Processing
PPO	Proximal Policy Optimization
PV	Principal Variation
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RLVR	Reinforcement Learning with Verifiable Rewards
UCI	Universal Chess Interface
VRAM	Video Memory

1 Introduction

The idea of machine intelligence has intrigued people for decades [58]. With the ideas of Alan Turing in the 20th century, people have started developing machines capable of doing indistinguishable things from human work. In recent years, this pursuit has culminated in the massive growth and popularity of Large Language Models (LLMs), which are already capable of passing the Turing test [24]. Following the release of GPT-3 in 2020 [5], language models have shown strong improvements across benchmarks and real-world applications. Today, the frontier of this technology has expanded even further, with agentic artificial intelligence (AI) systems beginning to automate workflows across various domains. In this work, we aim to push this boundary by improving the language model’s ability to generate structured outputs in the field of chess puzzle generation.

When measuring the success of these systems, creativity is often thought of as a key component of intelligence [8]. Although, modern language models are capable of creative work, such as creating art, writing stories or making lyrics, they struggle with coming up with truly new and unique items [62]. Some limitations also remain in generating structured outputs. Research suggests that LLMs struggle especially with rare formats [64] and that imposing formatting constraints can occasionally lead to a decreased reasoning performance [66]. These difficulties may cause problems in tasks that need the structure to be perfect, such as rendering html etc. In this study, chess puzzles serve as a playground, which requires both following the format of the problem exactly, while reasoning about the position.

Simultaneously to the rise of autoregressive language models, diffusion models have been successfully adapted to discrete sequences and language modelling tasks. Lately, discrete diffusion models that transform random vocabulary tokens into text have become popular [14], as they offer some advantages compared to autoregressive models. Such models perform worse than autoregressive models thus far, however, another discrete diffusion paradigm, masked diffusion [2, 38] offers comparable performance to traditional LLMs. Unlike autoregressive generation, which builds sequences token-by-token from left to right, diffusion models iteratively refine the entire sequence globally. Masked diffusion models transform a sequence of unknown mask tokens into structured text by iteratively unmasking the token sequence. This non-directional generation process offers a promising alternative for tasks that demand following strict constraints. We select a masked diffusion model for our task.

We use chess puzzle generation as a framework for improving the capabilities of language models. This domain requires the models to balance between creativity, reasoning and structure, as chess puzzles are generally very rare and delicate. Changing even one piece on the board may entirely change the puzzle to something completely different. We take inspiration from Feng et al. [17] and define puzzles as chess positions, which have a unique best solution, as well as are sufficiently counter-intuitive. With this definition, we observe that a notably low proportion of positions are puzzles even in the Lichess dataset [33]. Around 1.29% of the positions pass these conditions in the Lichess dataset.

To train our model for this task, we first employ supervised training on the

Lichess puzzles dataset [33]. Our model takes a list of themes as an input and is able to generate positions, which match the given theme. After supervised training, we observe that around 0.18% of the positions generated by our model are valid puzzles. To improve this further, we employ reinforcement learning, as we are able to use the puzzle criteria as the reward function. We are able to improve our model to generate puzzles around 0.37% of the time.

We present four main contributions from this research that build on [17]. Firstly, we verify the results of [17] by demonstrating the effectiveness of masked diffusion models for generating structured outputs, specifically in the context of chess puzzle generation. Secondly, we demonstrate that puzzle generation can be conditioned on specific tactical themes, which addresses a limitation in [17]. Thirdly, we demonstrate that the performance of masked diffusion models can be improved through reinforcement learning, despite the difficulty of the task. This contribution builds on the reinforcement learning (RL) framework originally designed for autoregressive LLMs in [17], which we adapt and modify for masked diffusion models. Lastly, we introduce a new auxiliary task, best move generation, and demonstrate that it can further improve puzzle generation quality. We hope that these contributions generalize to other natural language processing (NLP) tasks.

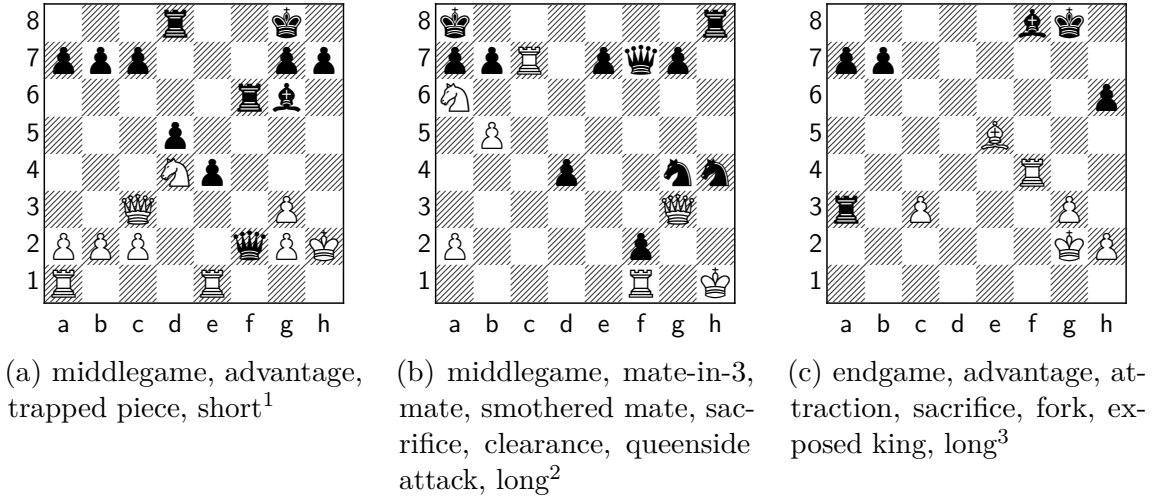


Figure 1: Three example puzzles from our model. In the subcaptions, we see the themes that are present in the puzzles and in the footnote, the themes that the model was conditioned with. White is to move in each puzzle.

We also present example puzzles generated by our model. In Figure 1, we see three different puzzles with unique themes. For each puzzle, the solution and the

¹Principal variation: e1e2 f2e2 d4e2, conditioning themes: middlegame, advantage, short, trapped piece

²Principal variation: c7c8 h8c8 g3b8 c8b8 a6c7, conditioning themes: long, middlegame, mate, sacrifice, attraction, mateIn3

³Principal variation: f4f8 g8f8 e5d6 f8g7 d6a3, conditioning themes: long, advantage, endgame, fork, exposed king, sacrifice, attraction

themes on which we conditioned the generation are in the footnote. These puzzles are filtered and hand-picked so that the uniqueness and counter intuitiveness checks passed, and the actual themes present in the puzzle matched the requested ones. More difficult puzzles are presented in Section 5.3.

The following outlines the structure of this thesis. In Section 2, we go through some important references in both chess and masked diffusion models. We describe how other works have previously approached chess puzzle generation, as well as present the technical algorithms we will use. In Section 3, we present the basic theory of masked diffusion models. Specifically, we describe how the forward and reverse processes are formulated and how one samples from a masked diffusion model. Additionally, we present the basics of reinforcement learning (RL) and the algorithms we use to eventually improve the model from a theoretical perspective. Lastly, we go through the basic chess terminology that is required to understand the premise of this work. In Section 4, we present the actual implementation of the tokenization procedure, supervised training, the RL reward function, and the RL implementation. Finally in Section 5, we describe how our model performs after supervised training and reinforcement learning. In the appendices, we give some additional detail for the algorithms, present additional results from the supervised and RL training phases and discuss the implementation details, such as the Stockfish version we use and where to obtain the source code of the work.

2 Literature review

The problem of chess puzzle generation has been studied quite extensively despite its limited practical applicability [17, 23, 22]. Chess platforms such as Chess.com and Lichess primarily generate puzzles by filtering large databases of human games to find positions with unique solutions. In the scientific community, more effort has been directed towards procedurally generating chess positions with desired properties.

One traditional approach to chess puzzle generation employs evolutionary search algorithms [17]. This process typically begins with a random board state and iteratively applies minor random modifications to improve the position. A fitness function evaluates each configuration, advancing the most promising candidates until a satisfactory puzzle emerges. Feng et al. [17] briefly discuss their initial trials with this technique, leaving a comprehensive exploration for future work. Although their primary methodology relies on deep learning, they suggest that a hybrid approach incorporating evolutionary search could be a good place for future research. Alternatively, one can employ an iterative refinement approach. Unlike evolutionary algorithms that maintain a large population of diverse positions, this method focuses on a single, pre-existing puzzle, incrementally applying small, deterministic modifications to optimize its aesthetic qualities [23, 22]. Notably, because the starting position is generally assumed to be a valid puzzle already, this functions more as a refinement technique than true generation.

In the field of chess puzzle generation, Feng et al. [17] demonstrate the viability of using deep learning models, in particular language models for chess puzzle generation.

They train four language models with different architectures using supervised training and then fine-tune the autoregressive model further using reinforcement learning. Their goal was to generate positions that humans would consider creative. They found that reinforcement learning substantially improved both the quality and quantity of the generated positions, ultimately producing positions comparable to award-winning compositions evaluated by master-level players and puzzle composers.

While autoregressive models have proven capable in this domain, the potential of diffusion large language models (dLLMs) remain comparably unexplored. Masked diffusion models, originally developed by [Austin et al. \[2\]](#) and later scaled up by others (see e.g., [38, 65]), offer tradeoffs over autoregressive architectures. In particular, they offer benefits in terms of efficiency during inference, as one can vary the amount of inference time compute depending on the task by using a different number of denoising steps. In addition, they are able to use bidirectional attention, which may offer benefits compared to autoregressive models. However, they take much longer to train [37].

A critical component of modern language model training is reinforcement learning, such as Reinforcement Learning from Human Feedback (RLHF) and Reinforcement Learning with Verifiable Rewards (RLVR) [48, 40, 30]. However, its application to discrete diffusion models is not as thoroughly studied. While [Black et al. \[3\]](#) provided a framework for applying reinforcement learning to continuous diffusion models, extending these methods to language modeling presents computational challenges due to the difficulty of evaluating exact token-wise transition probabilities within one forward pass, which is possible for autoregressive LLMs. Therefore, new methods have been proposed to mimic the work of [Zheng et al. \[69\]](#) and group sequence policy optimization (GSPO).

Recent works have proposed methods to approximate the likelihood of individual token sequences in dLLMs [39, 68, 54, 41], paving the way for RL techniques similar to those used in standard LLMs. For instance, ELBO-based sequence-level policy optimization (ESPO) [39] has been successfully applied to dLLM post-training. Compared to Denoising Diffusion Policy Optimization (DDPO) [3], ESPO works with a sequence-level Markov Decision Process (MDP), which allows one to use the evidence lower bound as a surrogate for the log probability of the sample. Instead, DDPO works with a step-wise MDP and exact transition probabilities offering unbiased gradient updates. Although DDPO is computationally less efficient than ESPO, meaning that the model update for each sampled trajectory takes quite a bit longer with DDPO, than with ESPO, DDPO allows the model to make updates for each sampled token individually (in reality DDPO works with all tokens sampled on a given denoising step, but this is still better than the whole sequence). This step-wise update could offer benefits in tasks, where individual tokens matter a lot. In summary, while RL has proven effective for autoregressive chess generation, its application to dLLMs remains an open challenge. To address this, we explore adapting DDPO for masked diffusion models, leveraging step-wise token updates to generate and optimize high-quality chess puzzles.

3 Background

In this section, we review the theory of the most important concepts used in this work. We begin in Section 3.1 by defining the masked diffusion model we employ as our base model. Then in Section 3.2, we proceed by describing the central concepts in reinforcement learning ending at Denoising Diffusion Policy Optimization (DDPO) [3], which we use as our chosen reinforcement learning algorithm. Finally, in Section 3.3, we give a brief overview of the game of chess.

3.1 Discrete Diffusion Models

Diffusion models are generative models that iteratively denoise inputs to generate items. Often, they are used in image [42, 44], video [21, 59] or audio generation [6, 27], where the samples are continuous. In these domains the model traditionally removes Gaussian noise on each denoising step. In recent years, many advances have been taken with diffusion models in discrete domains for instance by applying diffusion on continuous embeddings of the discrete data [31, 20, 35]. However, such models in the language modelling domain for instance, mostly do not perform as well as other models. Therefore, diffusion models that truly work on discrete data by iteratively modifying tokens, have been developed [2]. Specifically, masked diffusion models that iteratively transform mask tokens into normal tokens have become popular. Although, in the field of language modelling, these diffusion models can perform comparatively to LLMs, they require much more compute during training to achieve the same performance [37]. Nevertheless, masked diffusion language models might offer benefits during inference especially in efficiency [37].

In this work, we train a masked diffusion model to generate chess puzzles. We express each chess board as a sequence of tokens, in the same way one would express natural language. We describe the exact tokenization procedure in Section 4.1. For the model itself, we follow the work of Shi et al. [50]. We describe the masked diffusion model next.

3.1.1 Forward process

During the forward process, the tokens of a complete chess board are iteratively converted into mask tokens [MASK]. Similarly to [50], we denote the possible tokens by one-hot vectors $e_0, \dots, e_{m-1}$. Additionally, we denote the basis vector of the mask token by e_m :

$$e_0, \dots, e_{m-1}, e_m = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \in \mathbb{R}^{m+1}.$$

With this notation, we begin by examining the forward process in discrete time. Then, we follow [50] to generalize and take the limit as the number of discrete time steps approaches infinity.

Discrete-time forward process. To begin with, we divide the time interval $[0, 1]$ into T time steps. For each time $t \in [0, 1]$ and individual token y_t indexed by time, the transition from step $s(i) = (i - 1)/T$ to $t(i) = i/T$ is given by the following transition matrix:

$$Q_i = (1 - \beta_i)I + \beta_i \mathbf{1} e_m^\top \in \mathbb{R}^{(m+1) \times (m+1)}, \quad (1)$$

where $\mathbf{1} \in \mathbb{R}^{m+1}$ is the vector of ones and β_i determines the probability that the state $y_{s(i)}$ is transformed into the mask state. Each element $(Q_i)_{jk}$ tells the probability of moving from token j to token k with most of the transition probabilities being zero. From (1), we get the distribution for the state $y_{t(i)}$ in terms of the one-hot encoded state y :

$$q(y_{t(i)}|y) = \text{Cat}(y_{t(i)}; \bar{Q}_i^\top y), \quad \bar{Q}_i = \alpha_i I + (1 - \alpha_i) \mathbf{1} e_m^\top \text{ and } \alpha_i = \prod_{j=1}^i (1 - \beta_j),$$

where $\text{Cat}(x; p)$ is the categorical distribution with the event probabilities p .

To generalize this result to the entire token sequence, we use the factorization $\mathbb{P}[y_t^{\text{seq}}|y_s^{\text{seq}}] = \prod_{n=1}^N \mathbb{P}[y_t^{\text{seq},(n)}|y_s^{\text{seq},(n)}]$, which enables treating all transitions independently for each token [50]. Additionally, we choose the probabilities β_i such that $\alpha_1 \approx 0$ and $\alpha_0 = 1$. By these choices, we force the process to start from a fully unmasked state and converge to a sequence of only mask tokens.

Continuous forward process. Next, we generalize the previous notation to continuous time by taking the limit $T \rightarrow \infty$ of (1). We obtain the following equation [2, 50]:

$$\bar{Q}(t) = \alpha_t I + (1 - \alpha_t) \mathbf{1} e_m^\top, \quad \text{where } \alpha_t = \exp\left(-\int_0^t \beta(s) ds\right).$$

The function $\beta(s)$ is a continuous function such that $\beta_i = \frac{\beta(t(i))}{T}$, and must be defined by the user. Equivalently, one can choose the function α_t , which we call the masking schedule. A few common choices for the masking schedule include the linear schedule, the polynomial schedule, the geometric schedule and a cosine schedule. Although, Shi et al. [50] suggest using the linear schedule for training with a cosine grid for t during inference, we opt to use the linear schedule with a linear grid for t for both training and inference, as this choice is much more common in literature [50, 37]. The linear schedule is given by the equation:

$$\alpha_t = 1 - t.$$

One can find equations of the mentioned masking schedules and some experiments with the masking schedules in [50]. Finally, the authors of [50] derive an expression for the distribution of y_t conditioned on a one-hot encoded state y_s , $s < t$. This is notably important, as it allows one to obtain analytical expressions for the reverse process. The distribution of y_t is as follows:

$$q(y_t|y_s) = \text{Cat}(y_t; \bar{Q}(s, t)^\top y_s), \quad \bar{Q}(s, t) = \frac{\alpha_t}{\alpha_s} I + \left(1 - \frac{\alpha_t}{\alpha_s}\right) \mathbf{1} e_m^\top. \quad (2)$$

3.1.2 Reverse process

During the reverse process, we reverse the masking process and attempt to convert each masked token into the true token. Using (2), one can derive the following expression [50]:

$$\begin{aligned} q(y_s|y_t, y) &= \text{Cat}(y_s; \bar{R}^y(t, s)^\top y_t), \\ \bar{R}^y(t, s) &= I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} e_m(y - e_m)^\top \text{ and } s < t. \end{aligned} \quad (3)$$

Hence, if we know the original token sequence y that has no masking, we can compute the distribution of a step closer to the unmasked state. However, if we want to sample new data points using this process, y is unknown. To solve this problem, we introduce the denoising network $\mu_\theta(y_t, x) \in \Delta(V)^m$, which given a noisy state y_t and a prompt x , outputs a probability distribution over the vocabulary V for each masked state. Unmasked tokens remain the same with probability 1. Using this network and (3), we define the distribution for y_s as

$$p_\theta(y_s|y_t, x) = q(y_s|y_t, \mu_\theta(y_t, x)) = \text{Cat}(y_s; \bar{R}^{\mu_\theta(y_t, x)}(t, s)^\top y_t). \quad (4)$$

This formulation is called mean-parametrization, as the denoising network is attempting to predict the mean of y given the noisy input y_t and prompt x . Intuitively, (4) can be thought of as if y_t is a mask token, with probability $p_{\text{unmask}}^{(s)} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$, we sample a new token from $\mu_\theta(y_t, x)$, otherwise we keep the token from y_t . This allows us to sample new sequences from the network by iteratively converting mask tokens into other tokens via sampling from (4). We describe the sampling procedure in more detail in Section 3.1.4.

3.1.3 Evidence lower bound

To optimize the outputs of the denoising network, an optimization objective is needed. While traditional LLMs are optimized by maximizing the likelihood function, for masked diffusion models, efficient computation of the likelihood is intractable. Hence, with diffusion models one usually optimizes for the lower bound of the log marginal likelihood called the evidence [50, 2, 25]. If one optimizes the evidence lower bound (ELBO), the hope is that one also increases the likelihood. This usually happens in practice. Hence, this can be considered maximum likelihood optimization, although, one never explicitly optimizes the likelihood. In [50], the authors prove that for masked diffusion models, the evidence lower bound can be expressed in the following form:

$$\begin{aligned} \log p_\theta(y|x) &\geq \mathcal{L}_\theta(y|x) \\ &= - \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \mathbb{E}_{q(y_t|y)} \left[\sum_{i=1}^L \mathbf{1}[y_t^{(i)} = [\text{MASK}]] (y^{(i)})^\top \log \mu_\theta(y_t, x)^{(i)} \right] dt, \quad (5) \end{aligned}$$

where L is the sequence length, y is answer to the prompt x and y_t is the same answer with some tokens masked according to the masking schedule. One may note that the ELBO is just the weighted sum of the cross-entropy losses over the masked tokens integrated over time. This makes it easy to implement in practice.

Monte-Carlo integration To estimate the value of the ELBO, we employ Monte-Carlo (MC) sampling [13] for the inner expression. For a given t , the expectation can be estimated by sampling K different random masks, corrupting y with them, computing the sum of the appropriate cross-entropies and finally averaging over the K losses. The estimate of the expected value looks as follows:

$$\begin{aligned} \mathbb{E}_{q(y_t|y)} &\left[\sum_{i=1}^L \mathbf{1}[(y_t^{(k)})^{(i)} = [\text{MASK}]] (y^{(i)})^\top \log \mu_\theta(y_t^{(k)}, x)^{(i)} \right] \\ &\approx \frac{1}{K} \sum_{k=1}^K \sum_{i=1}^L \mathbf{1}[(y_t^{(k)})^{(i)} = [\text{MASK}]] (y^{(i)})^\top \log \mu_\theta(y_t^{(k)}, x)^{(i)}. \end{aligned}$$

In practice, we employ a 1-point MC estimate for the expectation. As the variance of the expectation estimate does not significantly affect the total variance of the ELBO estimate [41], we choose to use the available video memory (VRAM) to increase the batch size during supervised training.

In addition to the estimation of the expectation, during supervised training, we employ a 1-point MC estimate for the outer integral over t as well. Supervised training is a lot more stable than reinforcement learning and hence the variance of the ELBO estimate does not affect the training as much. We deem it more important to use all available VRAM to increase the batch size instead of using more MC samples. The final ELBO for supervised training has the form:

$$\mathcal{L}_\theta(y|x) = - \frac{1}{N} \sum_{j=1}^N \frac{\alpha'_{t_j}}{1 - \alpha_{t_j}} \frac{1}{K} \sum_{k=1}^K \sum_{i=1}^L \mathbf{1}[(y_{t_j}^{(k)})^{(i)} = [\text{MASK}]] (y^{(i)})^\top \log \mu_\theta(y_{t_j}^{(k)}, x)^{(i)}, \quad (6)$$

where $t_j \sim \mathcal{U}(0, 1)$ and $N = K = 1$.

3.1.4 Sampling

When sampling from the masked diffusion model μ_θ , we employ the reverse process described in (4) [50]. With ancestral sampling, we iteratively sample tokens from the conditional distribution $p_\theta(y_s|y_t, x)$, $s < t$. We begin by discretizing the interval $[0, 1]$ into T subintervals $\{t_i\}_{i=0}^T$, where T is the number of timesteps in the diffusion

process. The subintervals can be chosen in many ways, but we choose to use a uniform discretization. In each generation step, we condition the generation on the previously generated tokens and sample new tokens from a categorical distribution according to the masking schedule and the logits of the model. The sampling process is as follows [50]:

Algorithm 1 Sampling for masked diffusion models

```

Initialize the time grid  $\{t(i)\}_{i=0}^T \leftarrow \text{discretize}([0, 1])$ .
Initialize  $y_T$  by setting it to all mask tokens.
for  $j = T, T - 1, \dots, 1$  do:
     $t \leftarrow t(j)$ ,  $s \leftarrow t(j - 1)$ 
    for  $i = 1, 2, \dots, L$  do:
        if  $(y_t^{(i)}) = [\text{MASK}]$  then:
             $\log p_i \leftarrow \log \left( \frac{\alpha_s - \alpha_t}{1 - \alpha_t} \mu_\theta(y_t, x)^{(i)} + \frac{1 - \alpha_s}{1 - \alpha_t} e_m \right)$ 
            // Sample from  $\text{Cat}(p_i)$  using the Gumbel-max trick.
             $u^{(k)} \leftarrow \mathcal{U}([0, 1])$ 
             $y_s^{(i)} \leftarrow \text{argmax}_k \left( \log p_i^{(k)} - \log(-\log u^{(k)}) \right)$ 
        else:
             $y_s^{(i)} \leftarrow y_t^{(i)}$ 
        end if
    end for
end for

```

When we sample from the categorical distribution, we employ the Gumbel-max trick [28]. Shi et al. [50] note that using this trick yields considerably better sample quality than sampling from a categorical distribution using e.g. binary search on the cumulative distribution function (CDF) [50]. With this trick, we consider the log probabilities of all tokens, add Gumbel(0) distributed noise to each log probability and take the token with the highest sum. This procedure implicitly truncates small probabilities improving the sample quality compared to other theoretically equivalent sampling procedures.

To evaluate the depth of the model’s chess comprehension, we adopt a technique known as block diffusion [1]. We hypothesize that a model capable of explicitly generating the correct solution to a puzzle exhibits a more profound understanding of the underlying position. While Algorithm 1 can jointly generate both the board state and the optimal move simultaneously, we contrast this standard approach with a sequential sampling method that separates the generation process into two distinct denoising loops. In this block diffusion setup, the first loop exclusively denoises the position while strictly masking the tokens corresponding to the best move. Subsequently, the second loop denoises the optimal move conditioned on the fully unmasked board state. In Section 5.2, we empirically compare these two generation paradigms to determine whether jointly reasoning about the best move during the board state generation yields higher-quality puzzles compared to a sequential generation approach.

3.2 Reinforcement Learning

In this section, we work with Markov Decision Processes, which are defined as follows [52]:

Definition 3.1 (Markov Decision Process). A Markov decision process (MDP) is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \gamma, p_0)$, where $\mathcal{S}, \mathcal{A}$ are the state and action spaces and $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S} \times \mathbb{R})$ is the transition function of the environment characterized by transition probabilities $P(s', r | s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a]$ for states $s', s \in \mathcal{S}$, action $a \in \mathcal{A}$ and reward $r \in \mathbb{R} \setminus \{-\infty, \infty\}$. Moreover, $\gamma \in [0, 1]$ is a discount factor for the future rewards and $p_0 \in \Delta(\mathcal{S})$ is the distribution over the starting states.

Given an MDP $\mathcal{M}$, we consider an agent that explores $\mathcal{M}$. The agent takes actions according to a function $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, $\pi(s, a) = \mathbb{P}[A_t = a | S_t = s]$, which is called the policy, which might be parametrized by a parameter vector θ . Informally we denote this function as $\pi(a|s)$. The agent attempts to modify this policy to maximize the cumulative reward $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_k$ over a trajectory $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, \dots)$. In our setting, the trajectory always ends at a terminal state s_T with a reward r_T . A policy can be either explicitly defined or induced by a state-value function $V(s) = \mathbb{E}[G_t | S_t = s]$ or an action-value function $Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$ [52] via taking greedy actions.

3.2.1 Policy gradient theorem

In reinforcement learning, one often models the value function by function approximation, e.g. using a neural network, and uses an implicit greedy policy. In natural language processing, however, it is often more convenient to model a policy directly with function approximation, as the pretrained language model can serve as the policy¹. This way, the trained policy will remain stochastic, allowing sampling of completions. One can also avoid explicitly learning the value-function, which may affect the stability of the learning algorithm. Methods that learn the policy directly are called policy gradient methods. These methods rely on the policy gradient theorem, which yields gradients for the expected episodic reward.

Definition 3.2 (Expected episodic reward). For a given policy π_θ and a distribution of starting states p_0 , the function $J(\theta)$ is called the expected episodic reward and is given by the following equation:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{s_0 \sim p_0, \pi_\theta} [G_0] \\ &= \mathbb{E}_{s_0 \sim p_0} [\mathbb{E}_{\pi_\theta} [G_t | S_t = s_0]] \\ &= \mathbb{E}_{s_0 \sim p_0} [V_{\pi_\theta}(s_0)]. \end{aligned}$$

This definition allows us to formulate the policy gradient theorem as follows [53, 52]:

¹In this work, we will use the terms model and policy almost interchangeably.

Theorem 3.3 (Policy gradient theorem). *Let $J(\theta)$ be the expected episodic reward for a policy π_θ and let Q_{π_θ} be the action-value function for the same policy. Also let π_θ and Q_{π_θ} be differentiable with respect to θ and the gradients be bounded. Then we have*

$$\nabla J(\theta) = c \sum_s \mu(s) \sum_a Q_{\pi_\theta}(a|s) \nabla \pi_\theta(a|s) = c \cdot \mathbb{E}_{s \sim \mu, a \sim \pi_\theta} [Q_{\pi_\theta}(a|s) \nabla \log \pi_\theta(a|s)], \quad (7)$$

where μ is the on-policy (stationary) distribution under π_θ and c is a proportionality constant.

Originally proposed by Williams in [63], the policy gradient method allows one to optimize the policy parameters directly using gradient based optimization algorithms, which can be efficiently implemented using modern automatic differentiation engines. One of the original methods is the so-called REINFORCE with baseline [63], which serves as an inspiration for modern RL methods. In REINFORCE with baseline, one uses the following update formula [52]:

$$\theta_{t+1} = \theta_t + \alpha (G_t - b(S_t)) \nabla \log \pi_{\theta_t}(A_t|S_t),$$

where $b(S_t)$ is the baseline function, which is free as long as it does not depend on the action A_t . This update has the same expected value as in Theorem 3.3, as $\mathbb{E}[G_t|S_t, A_t] = Q_{\pi_\theta}(S_t, A_t)$ and $\mathbb{E}[\nabla b(S_t)] = 0$ [52]. If one chooses the baseline as $b(S_t) = V_{\pi_\theta}(S_t)$, one obtains the actor-critic algorithm [26], which serves as a basis for almost all modern RL algorithms used for LLMs. We describe the most common algorithms in the next section.

3.2.2 Proximal policy optimization

Proximal policy optimization (PPO) is a variant of the actor-critic method that prioritizes stability [47]. It introduces a clipping mechanism that allows one to do many model updates for each generation while controlling for stability. In addition, PPO uses importance sampling and often a KL-divergence penalty inspired by [46]. The PPO objective is defined using the importance ratio

$$\rho_t = \frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$$

and the advantage function $\hat{A}_t = Q(S_t, A_t) - V(S_t)$, which may be estimated in many different ways. The objective is given by the following equation:

$$L(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \quad (8)$$

where $\text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) = \max(\min(\rho_t, 1 + \epsilon), 1 - \epsilon)$ clamps the importance ratios to the interval $[1 - \epsilon, 1 + \epsilon]$.

This objective is designed to prevent model collapse. Since the samples generated by the old policy θ_{old} are only accurate close to the original policy, the gradients

become unreliable if the old and new policies are too different. Therefore, if the probability ratio improves the objective, the update is clipped to ensure stability. This makes the PPO objective a pessimistic bound of the previously used $\rho_t \hat{A}_t$ objective and hence more stable [47].

3.2.3 Denoising diffusion policy optimization

Denoising diffusion policy optimization (DDPO) is an RL algorithm designed for diffusion models [3]. In their work, Black et al. [3] define an MDP for a continuous diffusion models, which have a normal posterior distribution for each step. Although, our posterior distribution for each step is a categorical distribution, we can still take inspiration from their MDP formulation for our setup. We define the MDP for our problem as follows:

$$\begin{aligned} s_i &= (x_{t(i)}, i), \quad a_i = x_{t(i-1)}, \quad \pi_\theta(a_i | s_i) = p_\theta(x_{t(i-1)} | x_{t(i)}), \\ P(s_{i-1}, r_i | s_i, a_i) &= (\delta_{x_{t(i-1)}}, \delta_{i-1}, \delta_{r_i}), \quad p_0 = (x_{t(n_{\text{steps}})}, n_{\text{steps}}), \end{aligned}$$

where $x_{t(i)}$ is the sequence of partially masked tokens and a_i is the action of sampling a new sequence based on (4). The transition following the action is deterministic with the reward function being described in Section 4.2.4. Lastly, p_0 is the initial distribution of the MDP, which corresponds to the sequence with all mask tokens.

Due to the formulation of our problem, we further simplify the action probability $\pi_\theta(a_i | s_i)$. By using (4) for each independent token, we get the following:

$$\pi_\theta(a_i | s_i) = p_\theta(x_{t(i-1)} | x_{t(i)}) = (p_{\text{unmask}}^{t(i-1)})^{|M_i|} \cdot (1 - p_{\text{unmask}}^{t(i-1)})^{|K_i|} \prod_{j \in M_i} \mu_\theta(x_{t(i)})^{(j)}, \quad (9)$$

where M_i is the set of indices that transform from a masked token to an unmasked one during a transition and K_i is the set of indices that stay as masked tokens. Using this decomposition, we may note that the probability $p_{\text{unmask}}^{t(i-1)} = \frac{\alpha_{t(i-1)} - \alpha_{t(i)}}{1 - \alpha_{t(i)}}$ is a constant for both the reference model and the trained policy at any given step i . This allows the following simplification:

$$\begin{aligned} \rho_i &= \frac{\pi_\theta(a_i | s_i)}{\pi_{\theta_{\text{old}}}(a_i | s_i)} = \frac{(p_{\text{unmask}}^{t(i-1)})^{|M_i|} \cdot (1 - p_{\text{unmask}}^{t(i-1)})^{|K_i|} \prod_{j \in M_i} \mu_\theta(x_{t(i)})^{(j)}}{(p_{\text{unmask}}^{t(i-1)})^{|M_i|} \cdot (1 - p_{\text{unmask}}^{t(i-1)})^{|K_i|} \prod_{j \in M_i} \mu_{\theta_{\text{old}}}(x_{t(i)})^{(j)}} \\ &= \prod_{j \in M_i} \frac{\mu_\theta(x_{t(i)})^{(j)}}{\mu_{\theta_{\text{old}}}(x_{t(i)})^{(j)}}. \end{aligned}$$

Hence, for reinforcement learning, we can only compute the ratio of the tokens that were transformed from masked tokens to unmasked tokens. With this importance ratio, we can apply PPO described in Section 3.2.2 to our MDP.

To prevent the policy from collapsing and to encourage exploration, we augment the standard PPO objective $L(\theta)$ with a Kullback-Leibler (KL) divergence penalty

against the pretrained reference model π_{ref} and an entropy bonus. The final objective we seek to maximize is given by the following equation:

$$\mathcal{J}(\theta) = L(\theta) - \beta \mathbb{E}_i [\text{KL}(\pi_\theta \parallel \pi_{\text{ref}} \mid s_i)] + \gamma \mathbb{E}_i [H(\pi_\theta \mid s_i)], \quad (10)$$

where β and γ are hyperparameters controlling the strength of the KL penalty and the entropy bonus, respectively. In the subsequent sections, we detail how the step-wise KL divergence and entropy terms can be computed analytically.

To compute the PPO objective $L(\theta)$, an estimate of the advantage function $\hat{A}_i$ is required. Traditionally, this is achieved by training a separate critic network to estimate the state-value function. However, maintaining and training a value network alongside the policy network is computationally expensive and memory-intensive, particularly for large generative models. In [48], the authors propose a way to estimate the advantage-function in PPO using an MC estimate of the value-function. Because our MDP only yields a non-zero reward at the end of the diffusion generation process, the value function $V(s_i)$ can be effectively estimated using the empirical mean of the final rewards from a group of trajectories. Specifically, if we sample a group of G sequences and obtain final rewards $\{R_1, \dots, R_G\}$, the baseline value is approximated as $\mu^{(g)} = \frac{1}{G} \sum_{j=1}^G R_j$. The advantage for the j -th sample is then computed by standardizing the reward as $\hat{A}_j = R_j - \mu^{(g)}$. This Monte-Carlo approach completely eliminates the need for a critic network, drastically reducing the memory overhead while still providing stable and effective gradients for policy optimization.

3.2.4 Kullback-Leibler divergence and entropy

Kullback-Leibler (KL) divergence is an important part of most RL algorithms. Although KL divergence is not a metric, in general, it is a measure of a distance between two probability distribution. It is given exactly by the following formula:

$$\text{KL}(q \parallel p) = \sum_x q(x) \log \frac{q(x)}{p(x)} = \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right].$$

KL divergence is used in LLM post-training to prevent the model from drifting too far from the pretrained reference model. Traditionally, people believe that all knowledge of an LLM is gathered during supervised training. By incorporating a KL divergence penalty during reinforcement learning, the model retains more knowledge from the pretraining. Post-training is aimed to make the formatting of the answers better and to improve the reasoning capabilities of the model. KL divergence is also used to prevent reward hacking during RL, which happens when a model finds a solution to an RL problem, but bypasses the wanted behavior by using some loophole.

In autoregressive language models, computing the exact sequence-level KL divergence is computationally intractable due to the exponentially large space of possible sequence completions. Consequently, practitioners typically rely on MC estimators, which often suffer from high variance [45] or biased gradients [55]. In contrast, for masked diffusion models, the step-wise transition probabilities factorize conditionally independently across all tokens. While standard language models still

rely on Monte-Carlo estimates to compute this step-wise KL divergence due to their massive vocabulary sizes, our small chess vocabulary allows us to bypass sampling approximations entirely. As a result, we can compute the exact analytical step-wise KL divergence efficiently, yielding perfectly unbiased gradients for optimization.

In addition to preventing model drift, another central challenge in LLM post-training is model collapse, where the model converges to generating only a narrow set of responses that get high rewards. To mitigate this and actively encourage exploration, an entropy bonus is typically added to the training objective. Entropy is a measure of how much uncertainty a probability distribution has on average. A low entropy means that the probability distribution is quite concentrated, whereas, a high entropy means that a distribution is quite uniform. The entropy of a discrete probability distribution p is defined as:

$$H(p) = - \sum_x p(x) \log p(x) = -\mathbb{E}_{x \sim p}[\log p(x)].$$

Similarly to the KL divergence, while the sequence-level entropy remains intractable, our small vocabulary size enables the exact analytical computation of the step-wise entropy. We detail the derivations for both the exact step-wise KL divergence and entropy below.

KL divergence computation To compute the step-wise KL divergence, we use (4) and the conditional independence between tokens on each step. The KL divergence simplifies as follows:

$$\begin{aligned} \mathbb{KL}(\pi_\theta \parallel \pi_{\text{ref}} \mid s_i) &= \sum_{a_i \in \mathcal{A}} \pi_\theta(a_i \mid s_i) \log \frac{\pi_\theta(a_i \mid s_i)}{\pi_{\text{ref}}(a_i \mid s_i)} \\ &= \sum_{j=1}^L \mathbb{KL} \left(p_\theta \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \parallel p_{\text{ref}} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \right) \\ &= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \mathbb{KL} \left(p_\theta \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \parallel p_{\text{ref}} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \right) \\ &\quad + \underbrace{\sum_{x_{t(i)}^{(j)} \neq [\text{MASK}]} \mathbb{KL} \left(\underbrace{p_\theta \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right)}_{=\delta_{x_{t(i)}^{(j)}}} \parallel \underbrace{p_{\text{ref}} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right)}_{=\delta_{x_{t(i)}^{(j)}}} \right)}_{=0} \\ &= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \left(p_{\text{unmask}}^{t(i-1)} (\mu_\theta(x_{t(i)})^{(j)})^\top \left(\log \mu_\theta(x_{t(i)})^{(j)} - \log \mu_{\text{ref}}(x_{t(i)})^{(j)} \right) \right. \\ &\quad \left. + (1 - p_{\text{unmask}}^{t(i-1)}) \log \underbrace{\frac{1 - p_{\text{unmask}}^{t(i-1)}}{1 - p_{\text{unmask}}^{t(i-1)}}}_{=0} \right) \\ &= p_{\text{unmask}}^{t(i-1)} \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} (\mu_\theta(x_{t(i)})^{(j)})^\top \left(\log \mu_\theta(x_{t(i)})^{(j)} - \log \mu_{\text{ref}}(x_{t(i)})^{(j)} \right) \end{aligned}$$

$$= p_{\text{unmask}}^{t(i-1)} \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \mathbb{KL} \left(\mu_{\theta}(x_{t(i)})^{(j)} \parallel \mu_{\text{ref}}(x_{t(i)})^{(j)} \right) \quad (11)$$

Hence, similarly to the importance ratio in Section 3.2.3, we can almost fully compute the KL divergence by using the probability distributions given by the current model to the reference model, which is easy to compute as both distributions are categorical distributions.

To monitor the training process of LLMs, it is customary to plot the mean token-level KL divergence. In autoregressive language models, this is often approximated using the k_1 , k_2 or k_3 estimators [45]. In our masked diffusion setup, we obtain a similar estimate by computing the average of the exact step-wise KL divergences over the trajectory:

$$\overline{\mathbb{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{1}{L} \sum_{i=1}^T \mathbb{KL}(\pi_{\theta} \parallel \pi_{\text{ref}} \mid s_i), \quad (12)$$

This makes the KL divergences comparable to normal LLMs and hence, we monitor this quantity in Section 5.2.

Entropy computation Similarly to the KL divergence, we can compute the exact step-wise entropy of the policy. The entropy of a distribution over each step factors into the sum of the token-wise entropies due to the conditional independence of the token transitions. The step-wise entropy is computed as follows:

$$\begin{aligned} H(\pi_{\theta} \mid s_i) &= - \sum_{a_i \in \mathcal{A}} \pi_{\theta}(a_i \mid s_i) \log \pi_{\theta}(a_i \mid s_i) \\ &= \sum_{j=1}^L H \left(p_{\theta} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \right) \\ &= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} H \left(p_{\theta} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right) \right) + \underbrace{\sum_{x_{t(i)}^{(j)} \neq [\text{MASK}]} H \left(\underbrace{p_{\theta} \left(x_{t(i-1)}^{(j)} \mid x_{t(i)} \right)}_{=\delta_{x_{t(i)}^{(j)}}} \right)}_{=0} \\ &= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \left(- p_{\text{unmask}}^{t(i-1)} (\mu_{\theta}(x_{t(i)})^{(j)})^{\top} \log \left(p_{\text{unmask}}^{t(i-1)} \mu_{\theta}(x_{t(i)})^{(j)} \right) \right. \\ &\quad \left. - (1 - p_{\text{unmask}}^{t(i-1)}) \log(1 - p_{\text{unmask}}^{t(i-1)}) \right) \\ &= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \left(- p_{\text{unmask}}^{t(i-1)} (\mu_{\theta}(x_{t(i)})^{(j)})^{\top} \log \mu_{\theta}(x_{t(i)})^{(j)} \right. \\ &\quad \left. - p_{\text{unmask}}^{t(i-1)} \log p_{\text{unmask}}^{t(i-1)} \underbrace{(\mu_{\theta}(x_{t(i)})^{(j)})^{\top} \mathbf{1}}_{=1} - (1 - p_{\text{unmask}}^{t(i-1)}) \log(1 - p_{\text{unmask}}^{t(i-1)}) \right) \end{aligned}$$

$$\begin{aligned}
&= \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} \left(p_{\text{unmask}}^{t(i-1)} H(\mu_\theta(x_{t(i)})^{(j)}) + H_b(p_{\text{unmask}}^{t(i-1)}) \right) \\
&= p_{\text{unmask}}^{t(i-1)} \sum_{x_{t(i)}^{(j)} = [\text{MASK}]} H(\mu_\theta(x_{t(i)})^{(j)}) + |M_{t(i)}| H_b(p_{\text{unmask}}^{t(i-1)}), \tag{13}
\end{aligned}$$

where $H_b(\cdot)$ denotes the binary entropy function, $\mathbf{1}$ is the vector of ones and $|M_{t(i)}|$ is the number of masked tokens at step i . This result shows that the total entropy is a combination of the entropy of the model’s output distribution scaled by the unmasking probability, plus the uncertainty of whether the masked tokens will be unmasked at all.

Notably, the second term of the entropy depends solely on the fixed masking schedule and is therefore constant with respect to the model parameters θ . Consequently, it only affects the absolute magnitude of the entropy and does not influence its gradient during optimization. Thus, when an entropy bonus is employed during reinforcement learning, as discussed in Section 3.2.3, the gradients purely flow through the entropy of the network’s output distribution μ_θ .

Similarly to the KL divergence, we monitor the diversity of the model outputs by plotting the mean token-level entropy over the trajectory:

$$\overline{H}(\pi_\theta) = \frac{1}{L} \sum_{i=1}^T H(\pi_\theta \mid s_i). \tag{14}$$

By summing the exact step-wise entropy across the trajectory and dividing by the sequence length L , the total entropy cleanly decomposes into two parts. The first part is a sum over the entropies of the model’s output distribution and matches the traditional LLM mean token-wise entropy in expectation. However, the total value includes an additional fixed constant C arising from the binary entropy term:

$$C = \frac{1}{L} \sum_{i=1}^T |M_{t(i)}| H_b(p_{\text{unmask}}^{t(i-1)}). \tag{15}$$

To compute this constant analytically, we use the property from Section 3.1 that the expected fraction of masked tokens at step i is exactly $q_i = 1 - \alpha_{t(i)}$. This means the expected number of masked tokens is $|M_{t(i)}| = Lq_i$, and the unmasking probability is $p_{\text{unmask}}^{t(i-1)} = \frac{q_i - q_{i-1}}{q_i}$. By substituting these into the binary entropy formula $H_b(p) = -p \log(p) - (1-p) \log(1-p)$, we can expand the constant:

$$\begin{aligned}
C &= \frac{1}{L} \sum_{i=1}^T L q_i \left[-\frac{q_i - q_{i-1}}{q_i} \log \left(\frac{q_i - q_{i-1}}{q_i} \right) - \left(1 - \frac{q_i - q_{i-1}}{q_i} \right) \log \left(1 - \frac{q_i - q_{i-1}}{q_i} \right) \right] \\
&= \sum_{i=1}^T [-(q_i - q_{i-1}) (\log(q_i - q_{i-1}) - \log q_i) - q_{i-1} (\log q_{i-1} - \log q_i)] \\
&= \sum_{i=1}^T [-(q_i - q_{i-1}) \log(q_i - q_{i-1}) + (q_i - q_{i-1}) \log q_i - q_{i-1} \log q_{i-1} + q_{i-1} \log q_i] \\
&= \sum_{i=1}^T [-(q_i - q_{i-1}) \log(q_i - q_{i-1}) + q_i \log q_i - q_{i-1} \log q_{i-1}].
\end{aligned}$$

The last two terms form a telescoping sum evaluating to $q_T \log q_T - q_0 \log q_0$. Because the process starts fully unmasked ($q_0 = 0$) and ends fully masked ($q_T = 1$), this term is exactly zero. Substituting back $\alpha_{t(i)} = 1 - q_i$, the constant evaluates to the following:

$$C = -\sum_{i=1}^T (\alpha_{t(i-1)} - \alpha_{t(i)}) \log(\alpha_{t(i-1)} - \alpha_{t(i)}).$$

For the linear masking schedule with T steps, the difference $\alpha_{t(i-1)} - \alpha_{t(i)} = \frac{1}{T}$ is constant. We get the following baseline:

$$C = -\sum_{i=1}^T \frac{1}{T} \log \left(\frac{1}{T} \right) = \sum_{i=1}^T \frac{\log T}{T} = \log T.$$

Because this constant baseline is purely an artifact of the fixed diffusion masking schedule, it artificially inflates the raw entropy values. To ensure our logged metrics are comparable to other language models, we subtract this constant C from the total trajectory entropy before plotting. By explicitly removing the $\log T$ baseline, the monitored metric isolates the first term of (14), natively yielding the model’s exact mean token-wise vocabulary entropy. This makes the values of our logged entropy comparable to those monitored in traditional LLM literature.

3.3 Chess terminology

Although, chess is a widely known game, it is necessary to review its basic terminology. Chess is a board game played on an 8x8 grid of squares. A game is played between two players, who take turns moving their pieces with a goal of checkmating the opponent’s king [18]. At the start of the game, each player has 16 pieces consisting of a king, queen, two rooks, two knights, two bishops and eight pawns, with each piece having its own unique movement rules.

A game of chess ends in either a win for one of the players or a draw. A win can occur if a player checkmates the opponent’s king, that is, puts the king under attack such that no legal moves remain for the opponent [18]. A player can also win if the opponent resigns, which often happens before checkmate if a player is clearly

losing. A draw can occur in several ways, such as with stalemate, where no legal moves remain, but the king is not in check. A draw can also be claimed by a player if the same position is repeated three times, or if no capture or pawn move has been made in 50 consecutive moves [18]. A game is drawn automatically if a position is repeated five times or no captures or pawn moves have been made in 75 moves [18]. Lastly, a game ends in a draw if neither player has sufficient material to checkmate the opponent.

A chess position can be compactly described with the Forsyth-Edwards Notation (FEN) [7]. The FEN notation stores information about the board state, the side to move, castling rights, en passant target square, halfmove counter and fullmove counter. An example of a full fen string of the initial position after e2e4 is as follows:

rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
board side castling en passant halfmove fullmove

Although, the FEN notation captures most of the information about a chess position, for instance a three-fold repetition is not encoded.

Moves in a chess position can be represented in several ways, but in this thesis, we use the Universal Chess Interface (UCI) notation [10]. The UCI notation consists of the starting square, ending square and the possible promotion piece. Using this notation, we define the principal variation (PV) $pv(s)$ of position s as the sequence of best moves for either player from a given position. This sequence can be computed with a chess engine such as Stockfish [15], which is commonly considered to be the strongest chess engine.

One of the main functions of chess engines is to evaluate a given position. This is usually done in the centipawn metric, which indicates the amount of material or tactical advantage a player has in terms of the value of a pawn. For instance, if White is down a pawn, which typically results in a baseline evaluation of -100 centipawns, but has a forced sequence threatening checkmate compelling Black to give up a queen to avoid losing, the engine will look past the immediate material deficit and evaluate the position at roughly $+800$ centipawns. The chances of winning can be defined in many ways, but in this thesis, we use the definition of Lichess [9], which applies the logistic function to the centipawn evaluation as follows:

$$w(a|s) = \frac{2}{1 + \exp(-0.00368208 \cdot \text{eval}(a|s))} - 1, \quad (16)$$

where s is the board state, a is the move to be made, and $\text{eval}(a|s)$ is the centipawn evaluation of the position after the move a has been made. The evaluation is relative to the side to move meaning that if the side who makes the move a is winning, the evaluation is positive. Otherwise, the evaluation is negative. This makes $w(a|s) \in [-1, 1]$. A positive value means that the side making the move has the advantage and a negative value means the side making the move has a disadvantage. If the action leads to a forced checkmate, the winning chances are 1 if the side making a move wins and -1 if the side making a move loses.

Generally, the knight and bishop are considered 3 times as valuable as a pawn, with rooks being valued at 5 times a pawn and queens at 9 times a pawn [11]. The king is considered infinitely valuable as its loss results in a loss of the game. The exact relative values of the pieces can, however, vary depending on the position.

Lastly, chess positions are either tactical or positional. A tactical position is one, where the game is decided by a sequence of forcing moves, such as checkmate or winning material, whereas, a positional position is decided with more nuanced strategic advantages. A tactical position is solved with a tactic, which is a sequence of moves that leads to a win. Common tactics include e.g. forks, pins, skewers, discovered attacks, sacrifices and zugzwangs [12]. In this thesis, we focus on puzzles, which are usually solved using tactics.

4 Methodology

In this section, we describe the methodology of our work. We begin by describing how we represent the chess positions as token sequences in Section 4.1. Then, in Section 4.2, we describe how we define the chess puzzles we generate. Finally, in Sections 4.3 and 4.4, we describe the algorithms we use to train our models.

4.1 Tokenization

To generate new chess positions with deep learning models, one has to transform each puzzle into a format the model can understand. This is done in a process called tokenization. To achieve this, we take inspiration from [43, 17] and use the convenient fen notation to represent the board state. We discussed the basics of the fen notation in Section 3.3.

We tokenize the FEN by padding the representation to a fixed length of 76 tokens. The board consists of 64 tokens, each representing a square on the board. The remaining 12 tokens are used to represent the side to move (1 token), the castling rights (4 tokens), the en passant target square (2 tokens), the halfmove counter (2 tokens) and the fullmove counter (3 tokens). The example fen presented in Section 3.3 in a tokenized form looks as follows:

10, 8, 9, 11, 12, 9, 8, 10, 7, 7, 7, 7, 7, 7, 7, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 4, 2, 3, 5, 6, 3, 2, 4	, <u>14</u> , <u>16, 17, 18, 19</u> , <u>33, 23</u> , <u>37, 38</u> , <u>37, 37, 39</u>
board	side castling en passant halfmove fullmove

In total, we use a vocabulary of 52 tokens, where 13 tokens represent different pieces and an empty square and are only used for the board representation. The remaining token vocabulary is used to represent the side to move with two tokens, the castling rights with four tokens and one no castling token, the en passant target square with eight file, eight rank and one no en passant token. Finally, the counters are represented with ten tokens for the numbers and an additional padding token. In addition to the fen, we concatenate the tokenized version of the best move to the

token sequence. For this, we use the UCI notation [10] to represent the move, as discussed in Section 3.3. To represent the promotion, four last tokens are needed in the vocabulary. For the sequence length, we need five additional tokens to represent all possible moves, including promotions.

In addition to the 52 standard tokens, the vocabulary includes a special mask token [MASK]. As described in Section 3.1, this token is used throughout the diffusion process to represent an unknown state.

4.2 Defining chess puzzles

To be able to generate chess puzzles, and train the model with reinforcement learning, it is necessary to define which positions classify as puzzles. We follow the work of Feng et al. and define puzzles as positions, which have a unique solution and that are sufficiently counter-intuitive. Additionally, a position must be realistic, i.e. one that could reasonably arise during a real game of chess. In Sections 4.2.1, 4.2.2 and 4.2.3, we describe each criteria in more detail. In addition, we also describe the exact reinforcement learning reward function in Section 4.2.4.

4.2.1 Legality

For a chess position to be a valid puzzle, as a prerequisite, it must be legal. However, it is nontrivial to define what it means for a chess position to be legal. One possible definition of legality in chess is that a position is considered legal, if it can be reached from the initial position by a sequence of legal moves. This condition, however, is infeasible to check for general positions, the number of possible continuations from the initial position grows exponentially (discussion in [51]).

Instead of the previously described approach, in this work, we settle for a more heuristic definition of legality. We consider a position to be legal if it passes the following conditions: exactly one king for both colors, less than or equal to 16 pieces of which a maximum of 8 are pawns for both colors, no pawns on the first or the eight rank, possible castling rights, possible en passant square, moving player is not giving check, maximum two checking pieces and possible double check configuration. We denote the legality condition as $\mathbb{I}_{\text{legal}}(s)$. To check these conditions, we use the python-chess library [19]. Although, these conditions do not guarantee that a position is reachable from the initial position, they guarantee that there is nothing obviously wrong with the position. In addition, they are very efficient to check, as they do not require any search of the game tree.

4.2.2 Uniqueness

A second condition that a puzzle must satisfy is that it must have a unique solution. Following the work of Feng et al., inspired by the Lichess puzzler project [16], we check the uniqueness by recursively comparing the best and the second-best moves found by Stockfish on the principal variation. Although, the real uniqueness check has more details as described in Algorithm 2, the general idea is as follows: if the winning chances $w(a|s)$ from (16) of the player to move after the best move are significantly

higher than the winning chances of the player to move after the second-best move, we consider the position to have a unique solution. The uniqueness condition is formally denoted by $\mathbb{I}_{\text{unique}}(s)$.

Algorithm 2 Uniqueness metric

Require: Initial position s_0 , Engine, Threshold τ_{uni}

```

 $s \leftarrow s_0$ 
while  $s$  is not terminal do
   $a_{\text{best}}, a_{\text{second}} \leftarrow \text{Engine.analyze}(s)$ 
  if  $\text{sideToMove}(s) = \text{sideToMove}(s_0)$  then
    if  $w(a_{\text{best}}|s) - w(a_{\text{second}}|s) \leq \tau_{uni}$  then
      If  $s = s_0$ , return False, otherwise break
    end if
  end if
   $s \leftarrow \text{applyMove}(s, a_{\text{best}})$ 
end while
return True

```

While Algorithm 2 illustrates the core concept of the uniqueness metric, our practical implementation incorporates several additional constraints to guarantee uniqueness. During the recursive evaluation, we record the principal variation of s_0 and subsequently truncate it to ensure that the sequence concludes on the original player’s turn with a non-forced move. Furthermore, we explicitly reject positions that offer only one initial legal move, result in trivial one-move puzzles, or lead to a repetition of moves. We also require the original player to maintain a clear advantage of 200 centipawns throughout the variation. Finally, in forced checkmates, we relax the strict uniqueness requirement: if multiple mates-in-one are available, the sequence is still considered unique provided that the evaluation of the best move that is not a mate-in-one is sufficiently poor. The link to the implementation can be found in Appendix D.

4.2.3 Counter-intuitiveness

Although, legality and uniqueness are necessary conditions for puzzles, they are not sufficient. Many trivial positions, which can not be considered puzzles, satisfy the legality and uniqueness criteria. For instance, a checkmate in a material deficit or a recapture of a hanging piece, which was traded on the previous move both satisfy these conditions, but are clearly too easy to be considered puzzles. We address this in the same manner as Feng et al. and introduce a counter-intuitiveness metric, which quantifies the difficulty of a puzzle. We use the following definition:

$$\begin{cases} r_{\text{counter-intuitive}}(s) = 0.8 \cdot \frac{d_{\text{cp}} - 1}{50} - 0.1 \cdot \frac{v_{\text{capturedMaterial}}}{9}, \\ \mathbb{I}_{\text{counter-intuitive}}(s) = r_{\text{counter-intuitive}}(s) > \tau_{\text{cnt}}, \end{cases} \quad (17)$$

where d_{cp} is the critical depth of the Stockfish’s search, i.e. the lowest depth, where Stockfish first finds the eventual best move computed with the maximum search depth,

which has an upper limit of 50. $v_{\text{capturedMaterial}}$ is the value of the material captured on the best move. The material values of the pieces are described in Section 3.3.

Intuitively, the counter-intuitiveness metric is designed to penalize obvious moves while rewarding moves that require deeper thought. The material penalty ensures that moves capturing high-value pieces, such as taking a queen, receive lower scores, as they are typically the first candidate moves a human player would consider. On the other hand, a high critical depth indicates that the engine had to search extensively before settling on the best move, which often correlates with "quiet" positional moves or sacrifices that are not immediately obvious. Since the critical depth carries a significantly larger weight than the material penalty, it is the primary driver of the metric.

However, the counter-intuitiveness and uniqueness metrics are generally inversely correlated, which makes generating positions that satisfy both criteria difficult. When a position possesses a single, strong variation that satisfies the uniqueness criterion, Stockfish, being the best chess engine in the world, typically finds the best move with a shallow search, resulting in a low counter-intuitiveness score. Conversely, positions that yield a high critical depth usually feature multiple candidate moves that the engine must evaluate, which often causes the position to fail the uniqueness criterion. This opposing dynamic makes generating high-quality puzzles challenging, as positions that simultaneously satisfy both criteria are rare.

4.2.4 Reward function

We consider a legal position to be a puzzle if it satisfies both the uniqueness and counter-intuitiveness metrics. Empirically, this condition is able to filter out puzzles from for instance a list of positions from human games. However, it alone is not a sufficient reward function for reinforcement learning, as in theory, the model could output a single position that satisfies both criteria, and get high rewards. Additionally, the positions generated by the model are not guaranteed to be realistic. To address this, we take inspiration from Feng et al. and introduce realism- and diversity filters to our reward function.

Realism It can be argued that the educational value of a puzzle is strongly tied to its realism. Although, a puzzle that will never appear in a regular game could have an incredible solution, we think that the puzzle will offer more educational value to the solver if the position could have appeared in a regular game. Therefore, we use similar methods as Feng et al. to prevent the model from generating unrealistic positions.

Firstly, we apply a KL-divergence based penalty to discourage the model from generating positions with piece counts that deviate significantly from those in the training data. We discuss the computation of the KL penalty in Section 3.2.4. Although, Feng et al. use the KL penalty as a part of the reward function, we subtract it from the PPO loss function (8) as done by Ou et al.. We do this as it is more stable than to add it to the reward function.

Secondly, we use piece count thresholds to prevent the model from increasing the

piece counts above unrealistic levels. Although, it is possible to promote multiple pawns in one game, this is quite rare and hence we consider it unrealistic. For a position to pass the piece count check, the number of each piece must be at most the number of that same piece at the beginning of the game. The piece counts are described in Section 3.3. We denote the piece count condition with $\mathbb{I}_{\text{piece counts}}(s)$.

Theme matching Since we condition the puzzle generation on specific themes, we integrate a validation step into the reward function to ensure that the generated puzzles accurately reflect the requested themes. To formulate this check, we must first identify the themes present in any given generated position. For this purpose, we leverage the Lichess puzzler pipeline [16], which provides robust detection for the most important themes used in our work. This pipeline implements checks for a variety of puzzle characteristics and tactics, including, but not limited to puzzle length, checkmate type, and specific motifs such as pins and forks. Computing these themes with the Lichess puzzler requires the position to possess a unique solution, as identifying certain tactical motifs, such as a fork, necessitates iterating over the principal variation. The full list of supported themes is provided in Appendix A.

To condition the generation on specific themes, we penalize the model during reinforcement learning if the requested themes are not met. Although the actual procedure is slightly more complicated, we define a puzzle to match the requested target themes if $\mathcal{T}_{\text{req}} \subseteq \mathcal{T}_{\text{gen}}$, where $\mathcal{T}_{\text{req}}$ is the set of requested themes and $\mathcal{T}_{\text{gen}}$ is the set of themes computed by the Lichess puzzler for the generated puzzle. We define the exact criterion in Appendix A. This condition is represented by the indicator function $\mathbb{I}_{\text{theme}}(s)$, which is incorporated into our final reward function.

Diversity-filtering To prevent model collapse during reinforcement learning, we employ similar diversity-filtering mechanisms as Feng et al. [17]. The main goal is to not allow the model to start to generate only one position during reinforcement learning. Although, masked diffusion models generally memorize the training data less compared to autoregressive models [36], they will still collapse if no diversity filtering is performed. Therefore, as the main filtering mechanism, we use the Levenshtein distance to measure the similarity between positions and between solutions.

The Levenshtein distance is a distance metric between two strings [67]. It is defined as the number of single-character insertions, deletions or substitutions required to transform one string to another. Using the Levenshtein distance, we define the following two functions:

$$\begin{cases} d_{\text{fen}}(s, s') = \text{lev}(\text{fen}(s), \text{fen}(s')) \\ d_{\text{pv}}(s, s') = \frac{\text{lev}(\text{pv}(s), \text{pv}(s'))}{\max(|\text{pv}(s)|, |\text{pv}(s')|)} \end{cases} \quad (18)$$

where $\text{fen}(s)$ denotes the padded fen of position s without the half and full move counters, as described in Section 4.1. $\text{pv}(s)$ denotes the principal variation of position s and is described in Section 3.3. The PV distance is normalized by the maximum number of moves in a principal variation between the two strings. In reality, we

truncate the principal variations to one move during RL, whereas, when evaluating our models, we truncate to 6 moves.

We employ these distance metrics in two distinct diversity-filtering mechanisms. For intra-batch diversity, each position s generated during an RL step is compared against all other positions in the same batch $\mathcal{B}$. We first identify the position $s^* = \arg \min_{s' \in \mathcal{B}, s' \neq s} d_{\text{fen}}(s, s')$ that minimizes the board distance to s . The reward for s is zeroed out unless both its board and PV distances to s^* exceed their respective thresholds, τ_{fen} and τ_{pv} . This intra-batch filtering is formally defined by the indicator function $\mathbb{I}_{\text{intra}}(s) = \mathbb{I}[d_{\text{fen}}(s, s^*) \geq \tau_{\text{fen}}] \cdot \mathbb{I}[d_{\text{pv}}(s, s^*) \geq \tau_{\text{pv}}]$. Similarly, to prevent the model from collapsing to previously discovered solutions, we apply inter-batch diversity filtering using a replay buffer $\mathcal{D}$, which is described in more detail in Section 4.4. We sample a random subset $\mathcal{D}_{\text{sub}}$ from the buffer and find the closest position $s_{\text{inter}}^* = \arg \min_{s' \in \mathcal{D}_{\text{sub}}} d_{\text{fen}}(s, s')$. The reward for s is zeroed out if it is too similar to s_{inter}^* in terms of either the board or PV distance. This inter-batch diversity constraint is represented by the indicator function $\mathbb{I}_{\text{inter}}(s) = \mathbb{I}[d_{\text{fen}}(s, s_{\text{inter}}^*) \geq \tau_{\text{fen}}] \cdot \mathbb{I}[d_{\text{pv}}(s, s_{\text{inter}}^*) \geq \tau_{\text{pv}}]$.

In their work, [Feng et al. \[17\]](#) find significant model collapse problems even with intra- and inter-batch diversity filtering with Levenshtein distances. To remedy these problems, they apply entropy filtering and an entropy penalty. Although entropy collapse problems are not as prevalent in our setup, as we use the masked diffusion model, we still experiment with entropy filtering and an entropy penalty. For each position, we give a small entropy bonus described in Section 3.2.3. Additionally, we zero out the reward of positions, which have a lower entropy than the threshold τ_{ent} . This is denoted as $\mathbb{I}_{\text{entropy}}(s) = \sum_{i=1}^T H(\pi_{\theta} | s_i) \geq \tau_{\text{ent}}$. We describe the entropy computation procedure in more detail in Section 3.2.4. In addition, we use the step-wise entropy to monitor the training process. We found the metric to be useful because if the model collapsed to one position, the entropy dropped close to zero. If the entropy surged above 1, the number of piece-count violations or illegal positions started to increase.

Using this diversity filtering setup, we can finally present the reward function we use for reinforcement learning. Mostly, we follow the work of [Feng et al. \[17\]](#), however, with slight modifications as described previously. The final reward function we use is given by the following equation:

$$R(s) = \begin{cases} -2 & \text{if } \neg \mathbb{I}_{\text{legal}}(s) \\ \max(10 \cdot r_{\text{counter-intuitive}}(s), 0) & \text{if } \mathbb{I}_{\text{legal}}(s) \cdot \mathbb{I}_{\text{filter}}(s) \cdot \mathbb{I}_{\text{unique}}(s) \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

where $\mathbb{I}_{\text{filter}}(s) = \mathbb{I}_{\text{diversity}}(s) \cdot \mathbb{I}_{\text{piece counts}}(s) \cdot \mathbb{I}_{\text{theme}}(s)$ and $\mathbb{I}_{\text{diversity}}(s) = \mathbb{I}_{\text{entropy}}(s) \cdot \mathbb{I}_{\text{intra, fen}}(s) \cdot \mathbb{I}_{\text{intra, pv}}(s) \cdot \mathbb{I}_{\text{inter, fen}}(s) \cdot \mathbb{I}_{\text{inter, pv}}(s)$. Although, this reward function is zero quite often, we found less sparse reward functions to yield worse results than the multiplicative structure we ultimately chose.

4.2.5 Model evaluation criteria

To evaluate the generative models, we rely on the same criteria that define our reward function. Our primary performance metric is the *puzzle rate*, which is defined as:

$$\mathbb{I}_{\text{puzzle}}(s) = \mathbb{I}_{\text{legal}}(s) \cdot \mathbb{I}_{\text{unique}}(s) \cdot \mathbb{I}_{\text{counter-intuitive}}(s).$$

For the model evaluation, we sample n_{samples} positions with the model using Algorithm 1. We then compute the average puzzle rate over the generated samples. In addition, we monitor the legal rate and the uniqueness rate individually, and the counter-intuitiveness rate for puzzles that pass the uniqueness check.

To quantify novelty, we measure how much the generated puzzles differ from the training data and from each other. Specifically, we define two metrics based on (18): the *Lichess distance* (the average minimum distance to the closest position in a Lichess subsample) and the *self-distance* (the average minimum distance to the closest position within a generated subsample). These metrics are computed as follows:

$$d_{\text{Lichess}}(\mathcal{A}, \mathcal{B}) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \min_{b \in \mathcal{B}} d(a, b),$$

$$d_{\text{self}}(\mathcal{A}, \mathcal{A}) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \min_{b \in \mathcal{A} \setminus \{a\}} d(a, b),$$

where $A \subseteq \mathcal{A}$ is a subsample of the generated positions, $B \subseteq \mathcal{B}$ is a subsample of the Lichess dataset, and $d(\cdot, \cdot)$ represents either $d_{\text{board}}(\cdot, \cdot)$ or $d_{\text{pv}}(\cdot, \cdot)$. Ultimately, we consider the best-performing models to be those that achieve the highest puzzle rate, conditioned on maintaining sufficiently large Lichess and self-distances. This ensures that the models successfully generate high-quality puzzles without simply memorizing the training data or collapsing into a limited set of identical positions.

To evaluate the success of the conditional generation, we compute the *theme match rate*, which measures the proportion of generated puzzles that successfully satisfy the requested themes according to the matching condition defined in Section 4.2.4. By monitoring the theme match rate alongside the puzzle rate, we can quantify how well the model adheres to the conditioning constraints without compromising the overall quality of the puzzles.

4.3 Supervised training

We use supervised pretraining to initialize the masked diffusion model before introducing reinforcement learning. As demonstrated by Feng et al. [17], training a model solely via reinforcement learning is practically infeasible in this domain. Because the set of legal chess positions constitutes only a minuscule fraction of all possible token sequences, a randomly initialized model is extremely unlikely to generate even a single legal chess position, preventing any meaningful policy gradient updates. Therefore, a supervised initialization phase is essential to establish a baseline understanding of chess syntax and legal board states.

For the pretraining phase, we utilize the Lichess puzzle dataset [33]. This dataset is derived from the open Lichess database [32], which contains games played on the Lichess platform, and is processed using the open-source Lichess puzzler pipeline [16]. Each data point includes the FEN string of the starting position, the corresponding principal variation, and metadata such as themes and difficulty ratings. While the dataset also contains contextual information, such as the opening classification and the original game ID, we discard these features in our work. In total, our dataset comprises 5,600,086 unique positions, from which we randomly reserve 50,000 for the test set and allocate the remainder for training. Notably, the FEN provided in the dataset does not represent the starting position of the puzzle itself. Instead, it describes the position immediately prior to the first move of the principal variation, which is played by the opponent. Therefore, to obtain the true starting state of the puzzle, we apply the first move of the PV to the initial FEN and remove this move from the principal variation.

For the masked diffusion, we employ a transformer-based denoising network inspired by Ruoss et al. [43] and Feng et al. [17]. The network begins with an embedding layer that maps the tokenized sequence and a fixed positional embedding into a 1,024-dimensional space. We also apply linear projections to map the multi-hot encoded themes and difficulty ratings into the same embedding space. This input representation is processed by 16 attention blocks, followed by a linear classification head. Each attention block utilizes pre-normalization with layer normalization [56, 57], an 8-head self-attention mechanism, and a SwiGLU activation function [49, 56, 57] with a hidden dimension of $\frac{8d}{3}$, where $d=1,024$ is the model dimension. Conditioning on metadata is achieved via an 8-head cross-attention mechanism. No causal masking is applied in the self-attention blocks because the masked diffusion process naturally handles the decomposition of the joint probability distribution. The resulting model contains 268,892,160 trainable parameters.

The pretraining of the masked diffusion model follows a standard supervised learning setup. At each training step, we sample a batch of target puzzles, alongside their corresponding themes and difficulty ratings, from the dataset D . For each sample in the batch, a timestep t is drawn uniformly from the interval $[0, 1]$. We then corrupt the tokenized puzzles by sampling from the forward diffusion process. Finally, we compute the negative evidence lower bound using (6) and its gradient with respect to the model parameters, and update the weights using the AdamW optimizer. We repeat this training loop for a total of 2,000,000 steps. The complete procedure is detailed in Algorithm 3.

Algorithm 3 Supervised pretraining

Require: Dataset D
for $i = 1 \dots n_{\text{steps}}$ **do**

 Sample a batch of themes and ratings x and puzzles y from D

 For each sample in the batch, sample a timestep $t \sim \mathcal{U}(0, 1)$

 Corrupt each target y into y_t by sampling from $q_t(\cdot|y)$ via (1)

 Compute the loss $-\mathcal{L}_\theta(y|x)$ using the negative evidence lower bound in (6)

 Compute the gradient of the loss with respect to the model parameters w

 Clip the l_2 gradient norm of the parameters w

 Update the parameters w using the optimizer

Evaluate loss metrics and generate sample positions for validation

end for

To train the model, we configure the supervised learning process with a set of specific hyperparameters. We utilize a batch size of 1,024 and a weight decay coefficient of 10^{-4} . The optimization is performed using the AdamW optimizer with a learning rate schedule that features a linear warmup for the first 1,000 steps from 10^{-6} to 10^{-4} , followed by a cosine annealing schedule decaying to a minimum learning rate of 10^{-5} over the remaining steps [34]. To stabilize the updates, we clip the l_2 gradient norm of the model parameters to a maximum threshold of 1.0. For the noise schedule, we adopt a linear masking schedule. The training is executed on a cluster of eight Nvidia H200 GPUs.

4.4 Reinforcement learning algorithm

A primary objective of this work is to enhance the creativity and qualitative attributes of the masked diffusion model through reinforcement learning. To achieve this, we employ the DDPO algorithm, which is specifically designed for diffusion models. While the theoretical formulation of the DDPO objective was introduced in Section 3.2.3, here we detail the practical reinforcement learning setup tailored to our task. Because the reward function is quite sparse, meaning that the model gets nonzero rewards quite rarely, we must carefully balance the trade-off between policy collapse and over-regularization.

Modern actor-critic algorithms traditionally rely on a separate critic network to estimate the advantage function. However, for language modelling, as described in Section 3.2.3 people usually employ MC value estimates instead. Therefore, as proposed by Kool et al. [29], to compute the advantages $\hat{A}^{(i)}$, we employ the following unbiased Monte Carlo estimator:

$$\hat{A}^{(i)} = R(x, y^{(i)}) - \frac{1}{G-1} \sum_{j \neq i} R(x, y^{(j)}), \quad (20)$$

where G is the group size and R the reward function [29]. By evaluating a generation $y^{(i)}$ against the average reward of the remaining $G-1$ samples in the group, the estimator naturally centers the advantages with an unbiased baseline. To further

stabilize the gradients, we standardize these values by dividing the mean-centered advantages by the standard deviation of the rewards within the group. Consequently, the model is rewarded for generations that outperform the batch average and penalized for those that perform worse, while the standardization ensures the scale of the advantages remains consistent across varying reward distributions.

Our complete reinforcement learning procedure iterates over a predetermined number of training steps. In each step, we sample a batch of n_{batch} themes and difficulty ratings x , and generate corresponding positions y using algorithm 1. On each step of sampling, we store the log probabilities and tokens that were sampled. We then compute the rewards r for these generations using (19). Over μ inner optimization iterations, we compute the log probabilities of steps with the reference and the up-to-date models. Within these values, the KL divergence penalty (11), the entropy bonus (13), and the DDPO loss (10) are evaluated over the generated samples. We then update the model parameters using the AdamW optimizer. Finally, we record the relevant metrics for the current step. The full procedure is formalized in Algorithm 4.

Algorithm 4 Reinforcement learning algorithm

```

for  $i = 1 \dots n_{\text{steps}}$  do
  Sample  $n_{\text{batch}}$  random themes and ratings  $x$  from the dataset.
  Sample  $G$  positions  $y$  from the model with algorithm 1 and store the trajectories.
  Compute rewards  $r$  for  $y$  with (19) and estimate the advantage using (20).
  Compute  $\pi_{\text{old}}(y|x)$  and  $\pi_{\text{ref}}(y|x)$  with no gradient tracking.
  for  $j = 1 \dots \mu$  do
    Compute the new log probabilities  $\pi_{\theta}(y|x)$ .
    Compute the DDPO objective with (10).
    Maximize the DDPO objective via gradient ascent.
  end for
  Save the rewards and other metrics for monitoring.
end for

```

The reinforcement learning algorithm has many different hyperparameters. We use 6 groups of 16 samples each. This results in a total of 96 generated positions for each gradient update. We use a learning rate of $3 \cdot 10^{-5}$ and $\mu = 1$ gradient updates per generation. We observe that a larger μ results in the model collapse considerably easier. Additionally, we use a weight decay of 0.01. The KL- and entropy coefficients are chosen such that the KL divergence stays low, and the entropy grows over the training. We perform manual hyperparameter search for these coefficients in appendix C. The final values we ultimately chose were $\beta = 0.1$ and $\gamma = 0.2$. Lastly, to prevent model collapse, we employ intra- and inter batch distance regularization in the reward function as shown in (19). The board distance limits are both selected to be 6 units and the PV distance limits are 1. The entropy mask threshold is chosen as $\log 128 + 0.6 \approx 5.45$, where 128 is the amount of denoising steps in the sampling process. The training is performed with one NVIDIA H200 GPU.

5 Results

In this section, we present the main results of our model training and inference. We present the results of supervised pretraining in Section 5.1, and the results of the reinforcement learning in Section 5.2. Finally, we experiment with the final model’s ability to generate puzzles in Section 5.3.

5.1 Supervised results

After training our model with the setup described in Section 4.3, we obtain the loss curve shown in Figure 2. We see that the model learns the data well, as the test loss decreases steadily over the course of training. At around 1,000,000 training steps, however, we observe that the model starts to overfit slightly, as the test loss starts to increase. The lowest test loss is observed at 930,000 training steps. Because we store checkpoints of our model at intervals of 100,000 steps, we choose the model trained for 1,000,000 steps. While this latest training run exhibited no instability, we had observed instability and model collapse during some earlier training runs. We managed to solve this by incorporating clipping to the cross-entropy weights in the evidence lower bound (5) as done by Shi et al. [50] in their implementation. Lastly, comparing Figure 2 to Figure 3, we observe that the loss corresponds quite well with the real metrics we are interested in.

As established in Section 4.2.5, our primary evaluation metric is the puzzle rate, which measures the proportion of generated positions satisfying both the uniqueness and counter-intuitiveness criteria. In addition to this, we independently track the legal rate, uniqueness rate, counter-intuitiveness rate, and various diversity filtering metrics. Figure 3 illustrates the trajectory of these key metrics throughout the supervised pretraining phase. We evaluate each checkpoint saved at 100,000-step intervals by sampling 10,000 positions. The results demonstrate that while the model rapidly learns to generate legal positions, mastering the more complex, nuanced metrics requires significantly more training time. Notably, the uniqueness rate shows steady, continuous improvement. In contrast, the counter-intuitiveness rate, and by extension the overall puzzle rate, exhibits considerable variance. We attribute this volatility to the inherently high variance of the counter-intuitiveness metric, as Stockfish can often find correct solutions almost instantly. Finally, because the evaluation context is sampled directly from the training set, we observe no degradation in these metrics that would indicate overfitting.

To evaluate the supervised model, we present the metrics in Table 1, derived from a sample of 200,000 positions generated using refined inference parameters, specifically, an increased number of 256 denoising steps. The results demonstrate strong generalization capabilities, as the model achieves comparable performance regardless of whether the conditioning context originates from the training or test data distribution.

In addition to evaluating the model’s generalization, Table 1 also illustrates the impact of different sampling algorithms on puzzle quality. We investigate whether explicitly tying the model’s chess understanding to the generation process improves

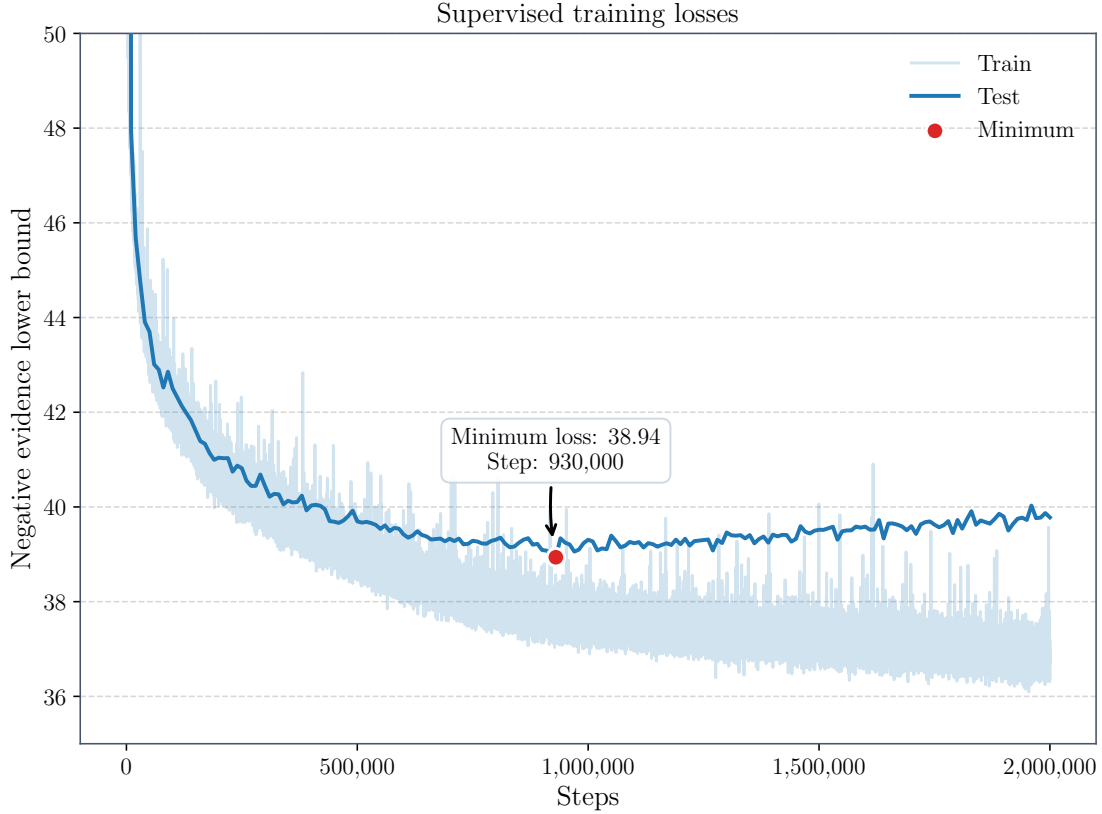


Figure 2: The negative evidence lower bound as a function of the training steps. After around 1,000,000, we observe clear overfitting.

its performance. To do so, we compare an approach where the position and the best move are sampled simultaneously against one where the best move is predicted only after the puzzle is generated. Interestingly, puzzle generation metrics measurably improve when the best move is generated simultaneously alongside the position, indicating that deeper understanding of chess directly enhances the model’s generative capabilities. To further quantify this underlying chess knowledge, Figure 4 plots the model’s accuracy in predicting the optimal move for valid puzzles over the supervised training. The results show that while the model correctly identifies the best move in most cases, the predictive accuracy stagnates early in the training process, exhibiting minimal improvement after the initial checkpoints.

Table 1 also presents the theme match rate, as defined in Section 4.2.5. For positions with unique solutions, the generated position aligns with the specified themes in over 85% of the cases. This proportion is substantial and suggests a strong understanding of various chess concepts. In fewer than 15% of instances, the model inadvertently generates a position with a unique solution that deviates from the required conditioning.

When contextualizing our results against existing literature, we find that our conditioning approach helps the model achieve similar uniqueness scores than the models trained by Feng et al. [17]. However, an exact comparison is complicated by

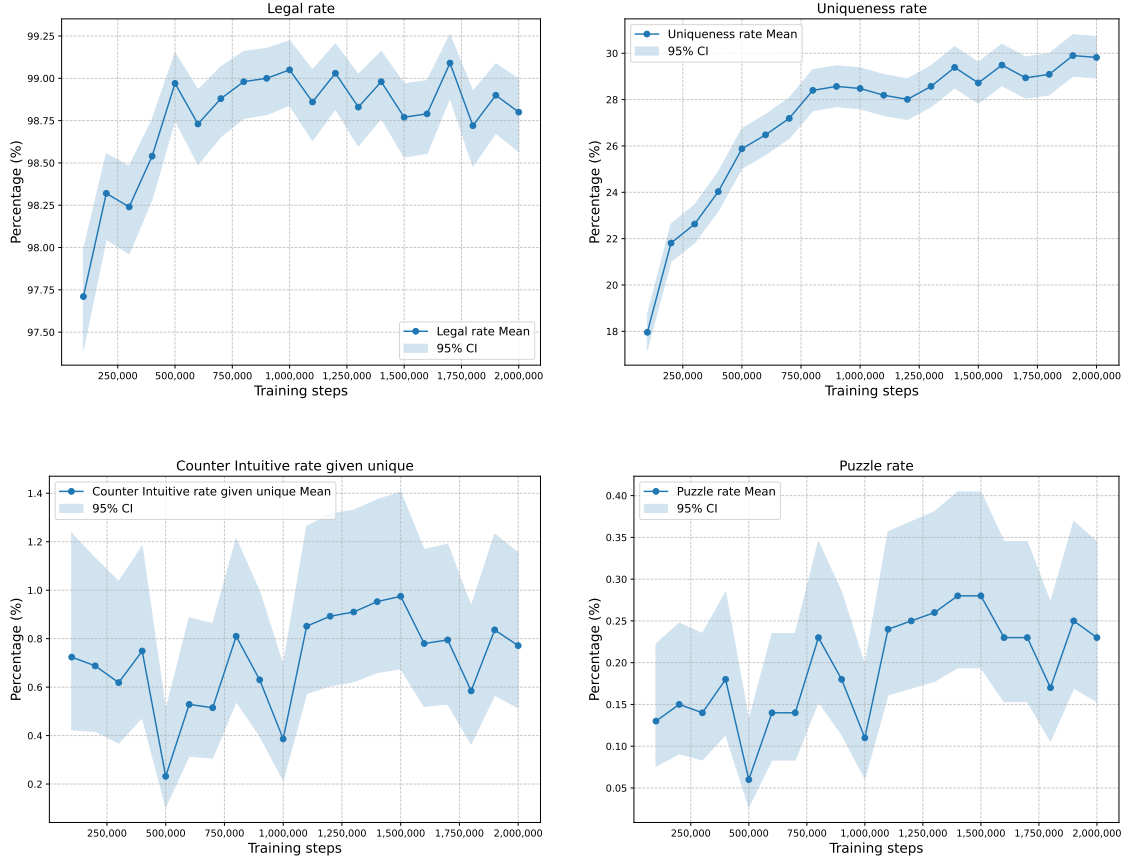


Figure 3: Evolution of the primary quality metrics with context from the training set during supervised pretraining. We omit the first data point, as it is identically zero for all metrics.

Table 1: The final metrics after supervised training with different context distributions. The sampling is performed with a temperature 1.0 and 256 denoising steps. The errors are computed as the 95% confidence intervals of the wanted proportion [4].

Model	Context	Sampling	Legal	Unique	Counter-intuitive given unique	Puzzle	Theme
SFT	Train	block	98.98 ± 0.044	27.10 ± 0.19	0.62 ± 0.018	0.17 ± 0.018	85.29 ± 0.18
SFT	Train	normal	98.99 ± 0.044	28.99 ± 0.20	0.63 ± 0.019	0.18 ± 0.018	86.33 ± 0.19
SFT	Test	block	98.97 ± 0.044	27.08 ± 0.19	0.63 ± 0.018	0.17 ± 0.018	85.53 ± 0.18
SFT	Test	normal	98.95 ± 0.045	28.75 ± 0.20	0.61 ± 0.018	0.18 ± 0.019	86.27 ± 0.19
RL	Test	normal	99.87 ± 0.01	25.28 ± 0.09	1.45 ± 0.012	0.37 ± 0.012	78.72 ± 0.08
Lichess puzzles			100.0 ± 0.0	83.58 ± 0.23	1.55 ± 0.070	1.29 ± 0.070	100.0 ± 0.0

differences in metric implementation. As demonstrated in Table 1, our implementations of the uniqueness and counter-intuitiveness checks are considerably stricter than those used by Feng et al. [17]. For instance, when evaluated on the same Lichess puzzle dataset, their implementation yields a 95.25% uniqueness rate and a 2.25% counter-intuitiveness rate (for positions with unique solutions), whereas our stricter evaluation yields significantly lower values. Despite these differences in absolute

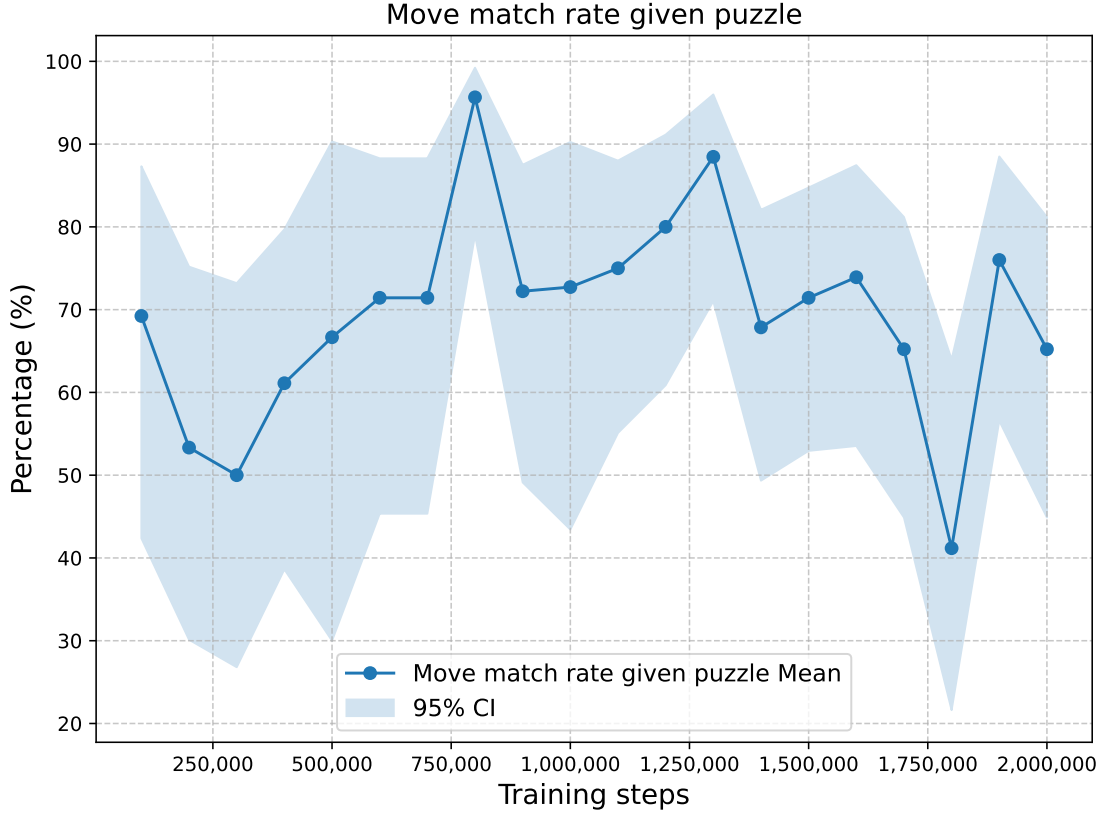


Figure 4: The evolution of the model’s ability to predict the best move for the positions, which pass the puzzle test.

measurement, the strong relative performance of our model supports the conclusion that explicit conditioning guides the model to generate more valid puzzles.

To assess the novelty of the generated positions, we utilize the distance metrics introduced in Section 4.2.5. Table 2 indicates that the baseline Lichess distribution exhibits a high average minimum board distance of 11.8827, corresponding to a difference of roughly 12 elements (pieces, side to move, castling rights, or en passant square) between positions. The average principal variation distance stands at 0.9496, reflecting a typical difference of one rank or file for either the start or target square of a move. Furthermore, Table 2 demonstrates that our trained model largely preserves these distance characteristics. While the model’s distances are slightly lower than those of the original Lichess puzzles, explicit conditioning effectively promotes diversity. Compared to the models proposed by Feng et al. [17], our generated positions exhibit comparable distances.

5.2 Reinforcement learning results

After completing the supervised pretraining phase, we fine-tune the model using the procedure described in Section 4.4. In this section, we evaluate the impact of this reinforcement learning phase on the model’s generative capabilities. We first examine

Table 2: The distance metrics after supervised training with different context distributions and sampling methods. The sampling is performed with a temperature 1.0 and 256 denoising steps. The self distances are computed with 40,000 samples and the Lichess distances with 100,000 samples. All generated positions are filtered so that we only compare positions that pass the uniqueness check.

Model	Context	Sampling	Board (Self)	Board (Lichess)	PV (Self)	PV (Lichess)
SFT	Train	block	11.5237	11.1493	0.8684	0.8894
SFT	Train	normal	11.5492	11.1601	0.8718	0.8900
SFT	Test	block	11.4987	11.1264	0.8655	0.8918
SFT	Test	normal	11.5497	11.1641	0.8723	0.8938
RL	Test	normal	15.0239	14.3468	0.8562	0.9533
Lichess puzzles			11.8827	-	0.9496	-

the training progression, focusing on the reward function and the stability of the training process. Next, we compare the primary performance metrics, namely the uniqueness, counter-intuitiveness, and puzzle rates, between the supervised baseline and the finetuned model. We also analyze the theme match rate to determine whether the model successfully adheres to the conditioning constraints without compromising puzzle quality. Finally, we investigate the effectiveness of our diversity and realism filters by analyzing the distance metrics of the generated positions, ensuring that the model we ultimately choose to use does not suffer from mode collapse.

In Figure 6, we see how most monitored metrics change during reinforcement learning. We observe that the reward steadily increases in Figure 6a. The increase in reward comes initially from the increase in the legal rate in Figure 6b and eventually from the increase in the counter-intuitive rate in 6e. Similar to [17], we observe that the uniqueness rate remains almost constant during the training, although with a sudden increase in the end when our model learns to exploit one specific puzzle type.

From Figures 6i-6l we observe that the model collapse happens at around 4,500 training steps, with the first signs being visible at around 3,500 steps. In the beginning, the training proceeds successfully and the diversity filtering is enough to prevent the model from collapsing. All distance metrics steadily increase and achieve a maximum at 3,000 steps. After collapsing, the model begins to generate mostly one counter-intuitive back rank checkmate with minor modifications that do not affect the solution. An example of such a position from White’s perspective is shown in Figure 5. Due to the model collapse, we select the checkpoint at 3,500 training steps for the final experiments, as the distance metrics still remain high, but the puzzle rate is considerably higher than the supervised model.

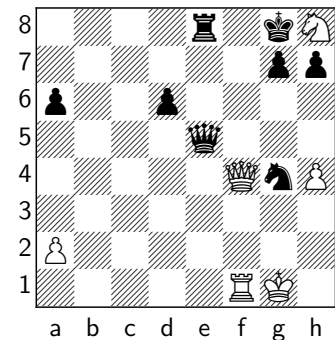


Figure 5: f4f7 g8h8 f7f8 e8f8 f1f8

For the final RL checkpoint at 3,500 training steps, we sample 1,000,000 positions and collect the same metrics as for the supervised model in Table 1. We choose to

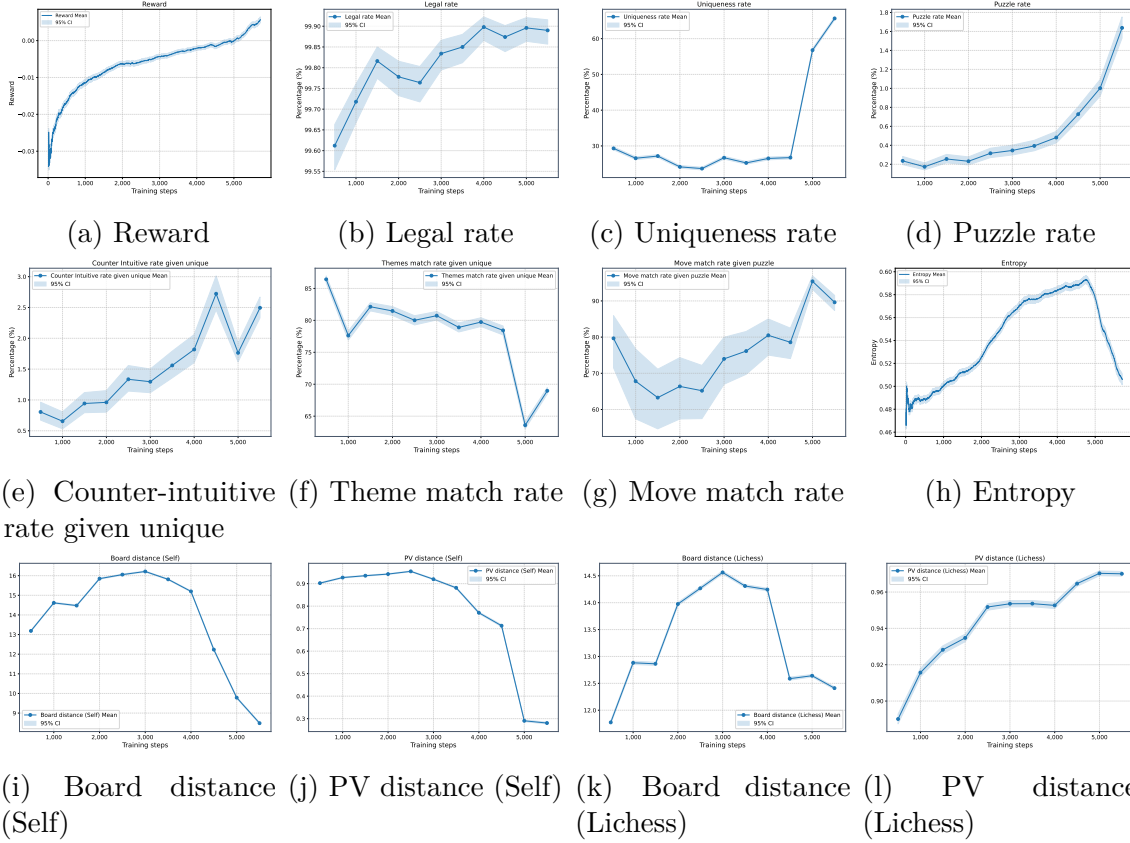


Figure 6: The training progress during reinforcement learning. We observe model collapse after around 4,500 steps. The self distances are computed with a subsample of 10,000 instead of 40,000 unique positions due to computational constraints, hence the values are larger than in Table 2. The shaded area represents the 95% confidence interval for the estimated value of the given metric.

employ normal sampling, as we saw additional benefits in terms of the uniqueness metric for the supervised model. Furthermore, we also condition the generation on themes sampled from the test set. We observe that the puzzle rate has approximately doubled, while a clear increase in the distance metrics can also be seen. Consistent with Figure 6, the uniqueness is slightly lower than before RL, while the counter-intuitiveness is approximately 2.3 times larger than previously. The increase in the puzzle rate can hence be primarily attributed to this rise in counter-intuitiveness. However, this improvement in puzzle quality comes at a slight cost to conditioning accuracy. The theme match rate slightly decreases compared to the supervised baseline, indicating that the RL objective shifts the model’s focus slightly away from adhering to the requested themes in favor of producing valid puzzles. Nevertheless, the total percentage of positions that pass the legality, uniqueness, counter-intuitiveness and the theme criteria increases from around $0.12\% \pm 0.02\%$ to around $0.26\% \pm 0.01\%$.

As seen in Figure 6, for the first 3,000 training steps, the distance metrics increase. By comparing SFT and RL in Table 2, we note that the board distances are considerably higher after reinforcement learning. The principal variation distance

to the Lichess dataset has also increased, although less than the board distances. The PV distance between the positions generated by the model has not meaningfully changed. This suggests that although the solutions to the puzzles might be similar to the ones before RL, by modifying other unimportant pieces, the model is able to make the positions more counter-intuitive. However, overall the model performs considerably better after reinforcement learning than before it.

5.3 Qualitative results

In this section, we bridge the gap between the quantitative metrics presented in Section 5.2 and the actual qualitative output of the final model. Mainly, we present and review a few puzzles generated by the model. After selecting the checkpoint we want to use, we generated 1,000,000 positions, 2,606 of which pass the legality, uniqueness, counter-intuitiveness, and theme criteria as described in Section 5.2. To further filter these positions, we sort them by the counter-intuitive value $r_{\text{counter-intuitive}}(s)$ in descending order. To select puzzles for this section, we manually review the most counter-intuitive puzzles and select the best ones. Although the author has experience playing chess and solving puzzles, it should be noted that the manual selection was conducted from the perspective of a casual puzzle enthusiast rather than a chess expert. Consequently, the chosen puzzles may reflect the author’s ability to understand chess, and a stronger chess player or puzzle composer may choose the puzzles differently. The chosen puzzles can be seen in Figure 7 with links to the closest matches from the Lichess dataset in the caption.

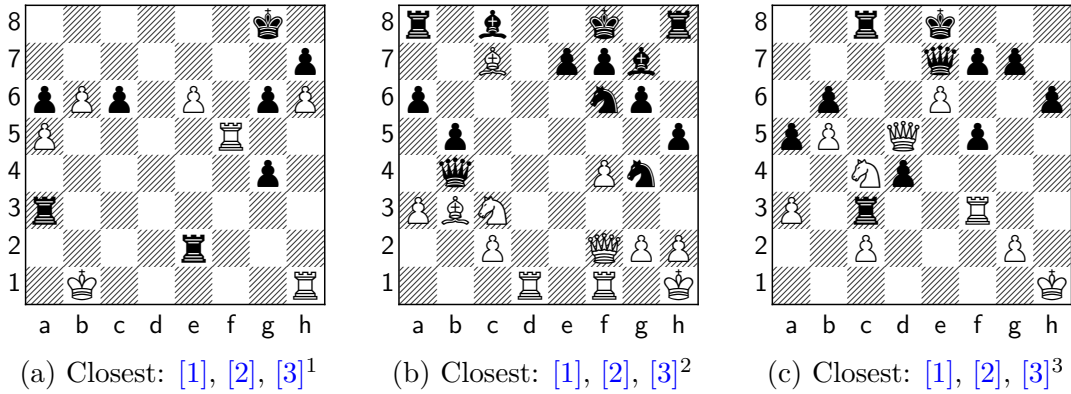


Figure 7: Three example puzzles.

In Figure 7a, we observe an endgame generated with the following themes: advantage, endgame, short, rook endgame. In the puzzle, White’s king is surrounded by both of Black’s rooks and is about to be checkmated if White does not act immediately. Additionally, White has many pawns just a few moves away from

¹Principal variation: f5b5 c6b5 b6b7

²Principal variation: f2d2 b4d6 c7d6

³Principal variation: e6f7 e8f8 c4e5 e7e5 d5e5

promotion. If given a chance, White would like to save its king and promote with the same move. The only move that wins for White is the incredible f5b5, which sacrifices the rook in two different ways to the Black pawns. The rook sacrifice has an idea that if it is accepted, the White pawn is guarded by the Black pawn and can easily promote. On the other hand, if it is not accepted, Black must sacrifice a rook for White’s pawn after it promotes, as Black is too slow to guard the promotion. The themes that are present in the generated puzzle are the following: endgame, advantage, advanced pawn, quiet move, sacrifice, rook endgame, short.

In Figure 7b, we see a puzzle generated with the following themes: crushing, middlegame, short. In the starting position, both White’s and Black’s queens are under attack and White has a chance to win Black’s queen with the best move. With the unfortunate placement of Black’s king, White can threaten back rank checkmate with f2d2 while avoiding Black’s attack on the queen. If White had a second move in a row, White would checkmate with d2d8 f6e8 d8e8 f8e8 d1d8. Black’s best response is to avoid checkmate by moving the queen with b4d6, where the queen is lost. If Black attempts a different move to avoid checkmate, White can capture Black’s queen with a3b4. White has another option that is on the edge of winning, but is much worse than the best move. White could check Black’s king immediately with d1d8, but this leads to a worse position where Black has chances to draw while down a rook but with a huge pawn majority. The themes that are present in the generated puzzle are middlegame, advantage, quiet move, short.

Lastly, in Figure 7c, we observe a puzzle conditioned on the following themes: advantage, endgame, short. Initially, the position seems good for Black, as White is down an exchange, but White has a winning resource. The best strategy for white is to capture the pawn with e6f7 to prepare to leverage the knight on c4. Black has only two legal moves as a response, but e7f7 walks straight into a fork of the king and the queen. Black’s best response is to shelter the king behind the pawn with e8f8. White has only one winning continuation by threatening another fork and a checkmate on g6 with c4e5. Black can attempt to save the game with a perpetual check on h4, but with White’s rook on f3, White can avoid the checks. The best response for Black is to avoid checkmate by sacrificing the queen for the knight with e7e5 and play the endgame with two rooks against a rook and a queen. The themes that are present in this puzzle are the following: endgame, advantage, advanced pawn, quiet move, attacking f2/f7, long.

6 Conclusion

In this work, we explored using a masked diffusion model for the generation of chess puzzles, building upon the framework proposed by Feng et al.. We initially trained a masked diffusion model using supervised training and subsequently enhanced its performance through reinforcement learning. A key aspect of our approach was conditioning the generative process on specific puzzle themes, effectively steering the generation toward desired puzzle types. After supervised training, our model successfully generated valid puzzles approximately 0.18% of the time, although the puzzle

rate varied considerably across different themes. By applying reinforcement learning, we more than doubled this success rate to around 0.37%, while simultaneously improving the diversity of the generated positions.

We presented four main contributions in this research. First, we demonstrated the effectiveness of masked diffusion models for generating strict, structured outputs. Second, we showed that chess puzzle generation can be accurately conditioned on specific tactical themes. Third, we established that the performance of masked diffusion models can be meaningfully improved through reinforcement learning, despite the inherent difficulty of the task. Finally, we demonstrated that integrating the auxiliary task of predicting the best move alongside the position enhances the generative capabilities of the model. While we focused specifically on chess puzzle generation and did not demonstrate generalization to other tasks in this study, our results indicate that masked diffusion models are highly capable and offer various avenues for improvement in structured generation tasks.

For the reinforcement learning phase, we employed the Denoising Diffusion Policy Optimization algorithm, an approach that is highly principled and theoretically grounded. To adapt this algorithm for our task, we formulated a Markov decision process following the methodology of [Black et al.](#). Crucially, our MDP was formulated step-wise, which contrasts with some of the latest research in discrete large language models [39, 41]. In this setup, the states correspond to the partially unmasked token sequences, while the actions define the generation of the subsequent partially unmasked sequence. The transitions between states are deterministic, and the rewards remain zero until the final step, at which point a non-zero reward is assigned based on the quality of the puzzle. To further stabilize the training process, we derived tractable expressions for the step-wise KL divergence and step-wise entropy. These expressions allowed a direct comparison to traditional autoregressive LLMs, and served as vital regularization tools to prevent model collapse and the generation of unrealistic positions.

Despite our results, we encountered several challenges and limitations during this research. For instance, while we hope our findings are broadly applicable, we have not yet demonstrated the generalization of these contributions to other natural language processing tasks. Additionally, the manual evaluation and selection of the generated puzzles were inherently constrained by the author’s level of chess knowledge. A stronger chess player or puzzle composer might offer a different perspective specifically regarding the evaluation of our final model. Furthermore, stability issues and model collapse during reinforcement learning posed significant challenges to our chosen algorithm. Relatedly, the observed decrease in the theme match rate during RL may stem from our specific theme match criterion, which is by no means the only way to define a successful match. It is possible that certain theme combinations inherently allow the model to generate puzzles more easily, causing it to converge toward generating positions that favor these "easier" themes. These limitations highlight clear avenues for further experimentation and study.

Looking ahead, numerous questions and directions for future research remain. During our experiments, we were unable to find a configuration where the recently published ESPO algorithm succeeded. Since algorithms that efficiently approximate

the importance ratio have shown benefits over DDPO [39, 60], discovering a viable ESPO configuration could further optimize the RL pipeline for this problem. Furthermore, applying similar setups to other NLP domains to showcase generalizability would significantly advance the research boundary. Lastly, experimenting with alternative puzzle generation frameworks, such as combining iterative algorithms and generative models within a chain-of-thought system [61], could not only improve chess puzzle generation but also pave the way for easier adaptation to complex problems in other domains.

AI disclosure

During this work, large language models (primarily Gemini 3.1 Pro and Gemini 3.5 Flash with both the web interface and Google Antigravity) were utilized to assist with programming tasks and text refinement. The author takes full responsibility for the entirety of this work, encompassing all experiments, software implementations, figures, tables, and written content. All output generated with AI assistance was manually reviewed and explicitly approved by the author. Furthermore, all architectural decisions in the software, as well as the choices regarding the thesis content, were made by the author, supported by the guidance of advisor Eric Malmi, supervisor Nuutti Hyvönen, and others.

References

- [1] Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tyEyYT267x>.
- [2] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/958c530554f78bcd8e97125b70e6973d-Paper.pdf.
- [3] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=YCWjhGrJFD>.
- [4] Lawrence D Brown, T Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.

- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- [6] Nanxin Chen, Yu Zhang, Heiga Zen, Ron J Weiss, Mohammad Norouzi, and William Chan. Wavegrad: Estimating gradients for waveform generation. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=NsMLjcFa080>.
- [7] Chess.com. Forsyth-edwards notation (fen). blog, 2020. URL <https://www.chess.com/terms/fen-chess>.
- [8] Simon Colton and Geraint A. Wiggins. Computational creativity: The final frontier? In *ECAI 2012 - 20th European Conference on Artificial Intelligence, 27-31 August 2012, Montpellier, France - Including Prestigious Applications of Artificial Intelligence (PAIS-2012) System Demonstration*, Frontiers in Artificial Intelligence and Applications, pages 21–26, Netherlands, 2012. IOS Press. ISBN 9781614990970. doi: 10.3233/978-1-61499-098-7-21. URL <http://ebooks.iospress.nl/volume/ecai-2012>. European Conference on Artificial Intelligence 2012, ECAI 2012 ; Conference date: 27-08-2012 Through 31-08-2012.
- [9] Lichess community. Lichess accuracy metric, 2026. URL <https://lichess.org/page/accuracy>. website.
- [10] Wikipedia contributors. Universal chess interface. Wikipedia, 2026. URL https://en.wikipedia.org/wiki/Universal_Chess_Interface.
- [11] Wikipedia contributors. Chess piece relative value. Wikipedia, 2026. URL https://en.wikipedia.org/wiki/Chess_piece_relative_value.
- [12] Wikipedia contributors. Chess tactic. Wikipedia, 2026. URL https://en.wikipedia.org/wiki/Chess_tactic.
- [13] Philip J. Davis and Philip. Rabinowitz. *Methods of numerical integration*. Academic Press, New York, 1984. ISBN 0-12-206350-3.
- [14] Google DeepMind. DiffusionGemma: An experimental open model that explores an exceptionally fast approach to text generation. blog, 2026. URL <https://deepmind.google/models/gemma/diffusionGemma/>.
- [15] The Stockfish developers. Stockfish. website, 2026. URL <https://stockfishchess.org/>.

- [16] Thibault Duplessis. Lichess puzzler. GitHub repository, 2020. URL <https://github.com/ornicar/lichess-puzzler>.
- [17] Xidong Feng, Vivek Veeriah, Marcus Chiam, Michael D Dennis, Federico Barbero, Johan Obando-Ceron, Jiaxin Shi, Satinder Singh, Shaobo Hou, Nenad Tomasev, and Tom Zahavy. Generating creative chess puzzles. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=TNZse5q2Tr>.
- [18] International Chess Federation FIDE. Fide handbook - laws of chess. blog, 2023. URL <https://handbook.fide.com/chapter/E012023>.
- [19] Niklas Fiekas. python-chess: a chess library for python. Python library, 2012. URL <https://github.com/niklasf/python-chess>.
- [20] Ishaan Gulrajani and Tatsunori Hashimoto. Likelihood-based diffusion language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=e2MCL6h0bn>.
- [21] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David Fleet. Video diffusion models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 8633–8646. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/39235c56aef13fb05a6adc95eb9d8d66-Paper-Conference.pdf.
- [22] Azlan Iqbal. A computer composes a fabled problem: Four knights vs. queen, 2017. URL <https://arxiv.org/abs/1709.00931>.
- [23] Azlan Iqbal. A computational method of optimizing chess compositions to enhance aesthetic appeal. In *2021 2nd International Conference on Artificial Intelligence and Data Sciences (AiDAS)*, pages 1–6, 2021. doi: 10.1109/AiDAS53897.2021.9574145.
- [24] Cameron R. Jones and Benjamin K. Bergen. Large language models pass a standard three-party turing test. *Proceedings of the National Academy of Sciences*, 123(21):e2524472123, 2026. doi: 10.1073/pnas.2524472123. URL <https://www.pnas.org/doi/abs/10.1073/pnas.2524472123>.
- [25] Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 21696–21707. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/b578f2a52a0229873fefc2a4b06377fa-Paper.pdf.
- [26] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*,

- volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.
- [27] Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. Diffwave: A versatile diffusion model for audio synthesis. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=a-xFK8Ymz5J>.
- [28] Wouter Kool, Herke Van Hoof, and Max Welling. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3499–3508. PMLR, 09–15 Jun 2019. URL <https://proceedings.mlr.press/v97/kool19a.html>.
- [29] Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 reinforce samples, get a baseline for free! In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=r1lgTGL5DE>.
- [30] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James Validad Miranda, Alisa Liu, Nouha Dziri, Xinxu Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Christopher Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=i1uGbfHHpH>.
- [31] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 4328–4343. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/1c31f49d288924b2cd395b0bdff68444-Paper-Conference.pdf.
- [32] lichess.org. Lichess open database, 2026. URL <https://database.lichess.org/>.
- [33] lichess.org. Lichess puzzle dataset, 2026. URL <https://database.lichess.org/#puzzles>.
- [34] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.
- [35] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Seo Shekhtman, and Kilian Q Weinberger. Latent diffusion for language generation. In *Thirty-seventh*

- Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=NKdtztladR>.
- [36] Xiaoyu Luo, Wenrui Yu, Qiongxiu Li, and Johannes Bjerva. Characterizing memorization in diffusion language models: Generalized extraction and sampling effects. In *ICML 2026 Workshop on Foundations of Deep Generative Models: Understanding Memorization, Generalization, and Reasoning*, 2026. URL <https://openreview.net/forum?id=1BZ10A1706>.
 - [37] Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=WNvwwK0tut>.
 - [38] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan LI. Large language diffusion models. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 50608–50646. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/48b383b24230e0e6e649d9c98dae4d8c-Paper-Conference.pdf.
 - [39] Jingyang Ou, Jiaqi Han, Minkai Xu, Shaoxuan Xu, Jianwen Xie, Stefano Ermon, Yi Wu, and Chongxuan Li. Principled RL for diffusion LLMs emerges from a sequence-level perspective. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=S5YeC91l1L>.
 - [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
 - [41] Kevin Rojas, Jiahe Lin, Kashif Rasul, Anderson Schneider, Yuriy Nevmyvaka, Molei Tao, and Wei Deng. Improving reasoning for diffusion language models via group diffusion policy optimization. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=JaqvessRBP>.
 - [42] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In

Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 10684–10695, June 2022.

- [43] Anian Ruoss, Grégoire Delétang, Sourabh Medapati, Jordi Grau-Moya, Li Kevin Wenliang, Elliot Catt, John Reid, Cannada A. Lewis, Joel Veness, and Tim Genewein. Amortized planning with large-scale transformers: A case study on chess. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 65765–65790. Curran Associates, Inc., 2024. doi: 10.52202/079017-2102. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/78f0db30c39c850de728c769f42fc903-Paper-Conference.pdf.
- [44] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo-Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=08Yk-n5l2A1>.
- [45] John Schulman. Approximating KL divergence. blog, 2017. URL <http://joschu.net/blog/kl-approx.html>.
- [46] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization, 2017. URL <https://arxiv.org/abs/1502.05477>.
- [47] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseek-math: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [49] Noam Shazeer. Glu variants improve transformer, 2020. URL <https://arxiv.org/abs/2002.05202>.
- [50] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 103131–103167. Curran Associates, Inc., 2024. doi: 10.52202/079017-3277. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/bad233b9849f019aead5e5cc60cef70f-Paper-Conference.pdf.

- [51] Chess stackexchange community. How to detect an illegal position? forum, 2023. URL <https://chess.stackexchange.com/questions/43340/how-to-detect-an-illegal-position>.
- [52] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [53] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.
- [54] Xiaohang Tang, Rares Dolga, Sangwoong Yoon, and Ilija Bogunovic. wd1: Weighted policy optimization for reasoning in diffusion language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=L2rfd2Czbj>.
- [55] Yunhao Tang and Rémi Munos. On a few pitfalls in KL divergence gradient estimation for rl, 2025. URL <https://arxiv.org/abs/2506.09477>.
- [56] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [57] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023. URL <https://doi.org/10.48550/arXiv.2307.09288>.
- [58] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236): 433–460, 1950.

- [59] Ruben Villegas, Mohammad Babaeizadeh, Pieter-Jan Kindermans, Hernan Moraldo, Han Zhang, Mohammad Taghi Saffar, Santiago Castro, Julius Kunze, and Dumitru Erhan. Phenaki: Variable length video generation from open domain textual descriptions. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=v0EXS39n0F>.
- [60] Guanghan Wang, Gilad Turok, Yair Schiff, Marianne Arriola, and Volodymyr Kuleshov. d2: Improving reasoning in diffusion language models via trajectory likelihood estimation. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=ldCiNVFt80>.
- [61] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=_VjQlMeSB_J.
- [62] Emily Wenger and Yoed N Kenett. Large language models are homogeneously creative. *PNAS Nexus*, 5(3):pgag042, 03 2026. ISSN 2752-6542. doi: 10.1093/pnasnexus/pgag042. URL <https://doi.org/10.1093/pnasnexus/pgag042>.
- [63] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, may 1992. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.
- [64] Jialin Yang, Dongfu Jiang, Tony He, Sherman Siu, Yuxuan Zhang, Disen Liao, Zhuofeng Li, Huaye Zeng, Yiming Jia, Haozhe Wang, Benjamin Schneider, Chi Ruan, Wentao Ma, Zhiheng Lyu, Yifei Wang, Yi Lu, Quy Duc Do, Ziyang Jiang, Ping Nie, and Wenhui Chen. Structeval: Benchmarking LLMs’ capabilities to generate structural outputs. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=buDwV7LUA7>. J2C Certification.
- [65] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.
- [66] Han Yuan, Yue Zhao, Li Zhang, Wuqiong Luo, and Zheng Ma. Quantifying the impact of structured output format on large language models through causal inference. In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1771–1795, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi: 10.18653/v1/2026.findings-eacl.91. URL <https://aclanthology.org/2026.findings-eacl.91/>.
- [67] Li Yujian and Liu Bo. A normalized levenshtein distance metric. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1091–1095, 2007. doi: 10.1109/TPAMI.2007.1078.

- [68] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=7ZVRlBFuEv>.
- [69] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025. URL <https://arxiv.org/abs/2507.18071>.

A Theme computation

To identify the tactical themes present in a generated position, we employ the Lichess puzzler pipeline [16], which iterates over the principal variation to detect specific themes. We group these themes into several distinct categories, which include the following: state of game, type of endgame, type of checkmate, length of checkmate, length of puzzle, winning states, and other tactics. Table A1 enumerates every specific theme we evaluate during the puzzle evaluation process. While the Lichess puzzles dataset [33] contains additional metadata tags, we exclusively focus on objective, position-based themes. We intentionally discard the other tags, such as “master vs. master”, as they cannot be verified from the board state alone.

During reinforcement learning and evaluation, we determine whether a generated puzzle successfully matches the requested conditions by comparing the generated theme set $\mathcal{T}_{\text{gen}}$ against the requested theme set $\mathcal{T}_{\text{req}}$. Rather than requiring a strict superset match $\mathcal{T}_{\text{req}} \subseteq \mathcal{T}_{\text{gen}}$, we use the specific groups for the check. The complete theme matching logic is detailed in Algorithm 5.

Category	Themes
State-of-game	Opening, Middlegame, Endgame
Type-of-endgame	Pawn endgame, Bishop endgame, Knight endgame, Rook endgame, Queen endgame, Queen rook endgame
Type-of-checkmate	Mate, Back rank mate, Boden mate, Smothered mate, Hook mate, Double bishop mate, Arabian mate, Dovetail mate, Anastasia mate
Length-of-checkmate	Mate in 1, Mate in 2, Mate in 3, Mate in 4, Mate in 5
Length-of-puzzle	One move, Short, Long, Very long
Winning	Crushing, Advantage
Other	Hanging piece, Fork, Interference, Kingside attack, Zugzwang, Exposed king, Skewer, Pin, Quiet move, Discovered attack, Sacrifice, Deflection, Advanced pawn, Attraction, Promotion, Queenside attack, Defensive move, Attacking f2/f7, Clearance, Intermezzo, Equality, Trapped piece, X-ray attack, Capturing defender, Double check, En passant, Castling, Underpromotion

Table A1: The complete list of themes supported by the Lichess puzzler pipeline, grouped by their respective categories.

Algorithm 5 Themes match

Require: Requested themes $\mathcal{T}_{\text{req}}$, Generated puzzle themes $\mathcal{T}_{\text{gen}}$
Require: Category sets $\mathcal{S}_{\text{state}}$, $\mathcal{S}_{\text{endgame}}$, $\mathcal{S}_{\text{mate_len}}$, $\mathcal{S}_{\text{mate_type}}$, $\mathcal{S}_{\text{other}}$

```

if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{state}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
  return False
end if
if "endgame"  $\in \mathcal{T}_{\text{req}}$  then
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{endgame}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
end if
if "mate"  $\in \mathcal{T}_{\text{req}}$  then
  if "mate"  $\notin \mathcal{T}_{\text{gen}}$  then
    return False
  end if
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{mate\_len}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{mate\_type}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
else
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{other}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
end if
return True

```

B Supervised training

In addition to the training metrics presented in Figure 3, we also monitor the model’s performance during supervised training by evaluating generated samples conditioned on themes drawn from the held-out test set. These test-set evaluation metrics are illustrated in Figure B1. We observe that while the legal rate and uniqueness rate saturate after approximately 1,000,000 training steps, the counter-intuitiveness and overall puzzle rates converge significantly faster. However, the inherent variance in the counter-intuitiveness metric makes identifying the precise convergence point of the puzzle rate challenging. We observe that the performance of these metrics do not decrease similarly to the test loss in Figure 2. We still select the checkpoint at 1,000,000 training steps, as the metrics do not improve with more training.

To further benchmark our the performance of our model, we additionally sample from the model with a lower temperature of 0.1 B1. We notice that with the almost greedy sampling, the uniqueness is considerably improved, whereas, the counter-intuitiveness is not. Furthermore, the difference between the move sampling is further

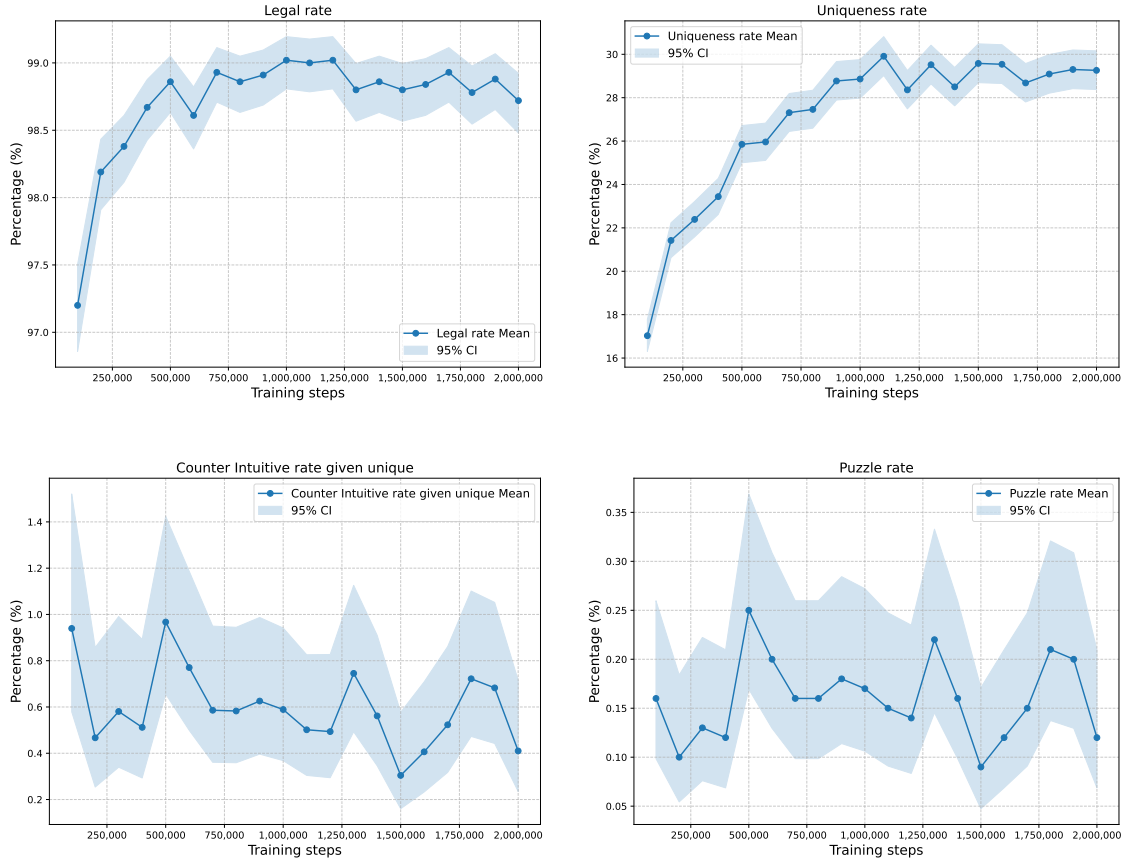


Figure B1: Evolution of the primary quality metrics with context from the testing set during supervised pretraining.

amplified, as if we sample the move after sampling the position, the uniqueness is considerably lower than if the move was sampled simultaneously to the position. We conclude that the best move prediction improves the model performance meaningfully. This improvement comes at a cost to the distance metrics though, as the board distance to the Lichess dataset is around 6.91 and the PV distance is 0.80 when sampling with a low temperature.

Table B1: The final metrics after supervised training with different context distributions. The sampling is performed with a temperature 0.1 and 256 denoising steps.

Context	Sampling	Legal	Unique	Counter-intuitive unique	Puzzle	Theme
Train	block	98.04 ± 0.27	35.20 ± 0.93	0.50 ± 0.081	0.18 ± 0.081	90.18 ± 0.90
Train	normal	99.44 ± 0.14	42.01 ± 0.96	0.63 ± 0.099	0.26 ± 0.099	92.75 ± 0.94
Test	block	98.33 ± 0.25	34.89 ± 0.92	0.50 ± 0.081	0.18 ± 0.081	89.98 ± 0.90
Test	normal	99.45 ± 0.14	42.37 ± 0.96	0.48 ± 0.088	0.21 ± 0.088	92.90 ± 0.95
Lichess puzzles		100.0 ± 0.0	83.58 ± 0.23	1.55 ± 0.070	1.29 ± 0.070	100.0 ± 0.0

C Reinforcement learning

C.1 Additional results

In this section, we present a few additional results on the reinforcement learning. We begin our experiments by training a model with using only the legality, uniqueness and counter-intuitiveness as the components of the reward. In Figure C1, we observe the results of such a training run. We observe that the model collapses after around 1700 training steps. For this run, we use a learning rate of $3 \cdot 10^{-5}$ with $\mu = 2$ gradient updates per batch and 32 groups of 6 generations with the same themes. Compared to the results of [Feng et al.](#), we see that the masked diffusion model is more resistant to overfitting compared to an LLM and takes a lot longer to collapse. This is supported by the findings of [Nie et al.](#), although our results are for reinforcement learning, whereas their results concern pretraining.

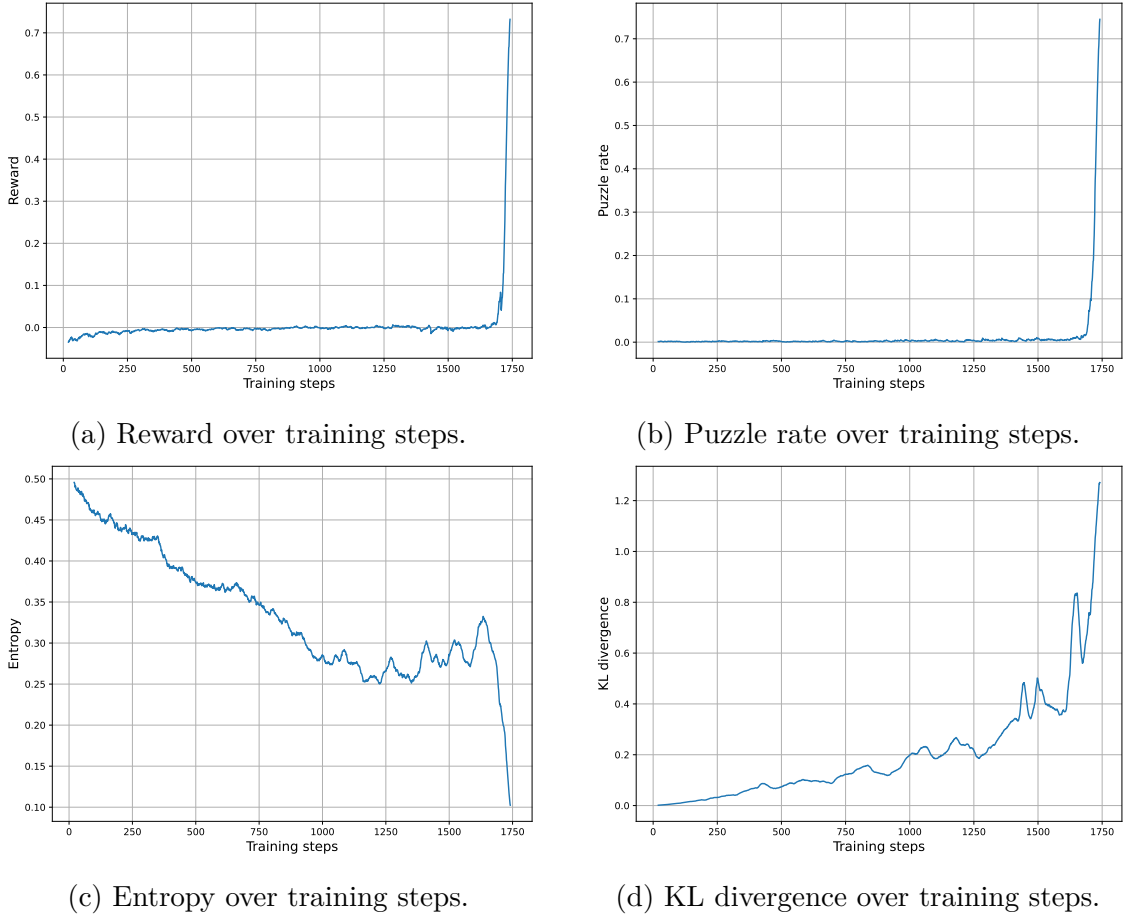


Figure C1: An illustration of DDPO training with just legality, uniqueness and counter-intuitiveness based reward. We observe model collapse at around 1700 training steps.

To combat model collapse, we introduce many diversity filtering mechanisms

inspired by [17]. We introduce the methods described in Section 4.2.4 and perform manual hyperparameter search for a good value of the entropy coefficient γ . In Figure C2 we observe that with a high entropy bonus coefficient $\gamma = 0.3$, the task is too difficult for our model to learn, but with a lower entropy bonus coefficient $\gamma = 0.03$ or $\gamma = 0.06$, the entropy slowly decreases.

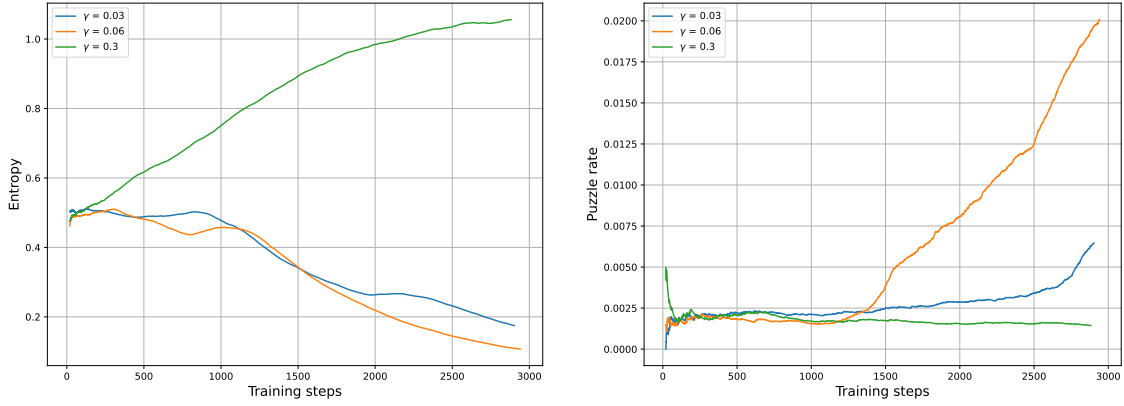


Figure C2: The training progress with different values of γ . We observe, that the two lower values overfit, whereas, the larger one does not learn.

In Figures C3, we present the distributions of themes in the lichess dataset as well as the generated positions. In Figure C4, we present the counter-intuitive rates for each of these themes. We notice that it is considerably more difficult to generate counter-intuitive positions. For instance, although mate-in-one puzzles are quite common, no counter-intuitive mate-in-ones are present in the Lichess dataset or the generated positions.

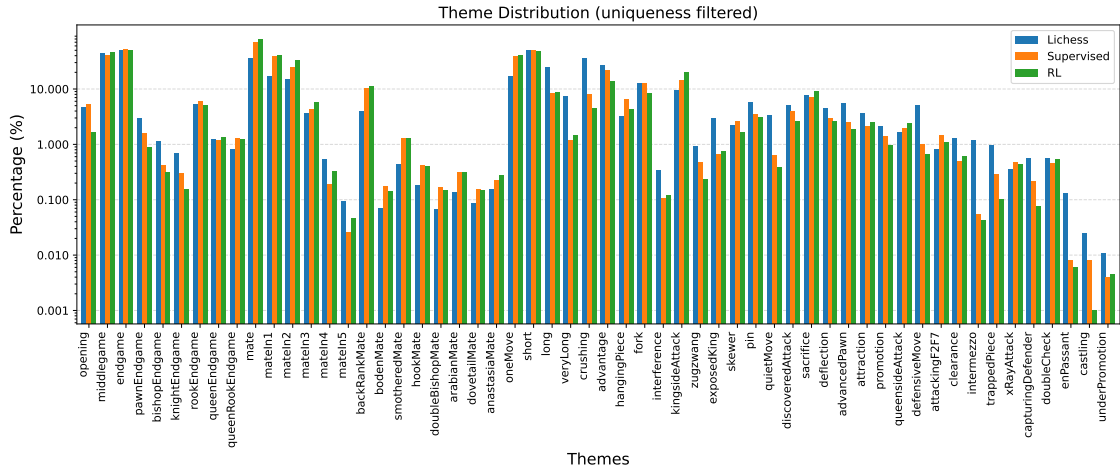


Figure C3: The distribution of themes for the Lichess puzzles dataset, our supervised model and the final model. All positions are first filtered for positions with a unique solutions and the percentages are computed accross the filtered dataset.

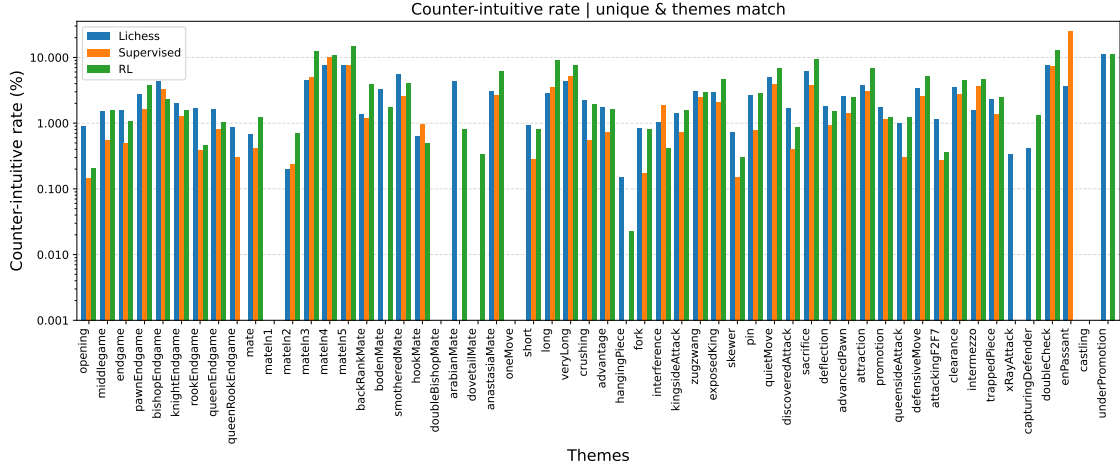
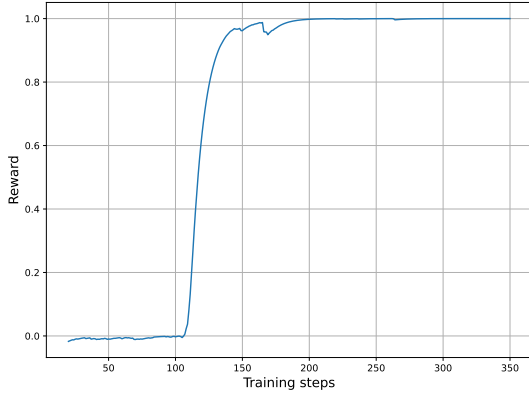


Figure C4: The counter-intuitive rate for the Lichess puzzles dataset, our supervised model and the final model across all themes. All positions are first filtered for positions with a unique solution and so that the themes match the ones the model was conditioned on, and the percentages are computed across the filtered dataset.

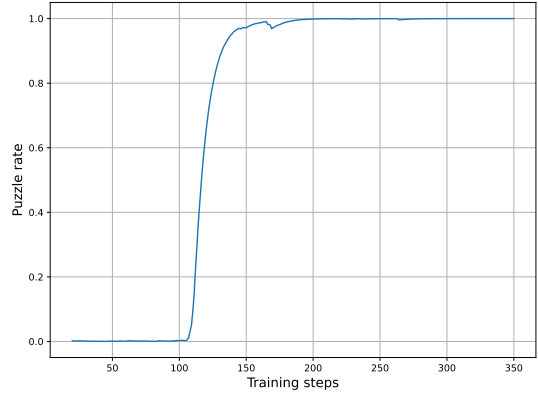
C.2 ELBO-based Sequence-level Policy Optimization

Before deciding to use DDPO, we attempt to use the ELBO-based sequence-level policy optimization (ESPO) algorithm [39]. ESPO is an algorithm specifically designed for masked diffusion language models, which led us to consider it first. Although we found some promising results, we were unable to get the model to learn effectively. In Figure C5, we see that we are able to make the model find a puzzle, which produces high rewards, however, after introducing diversity filtering and other reward components, the model either marginally improved without meaningful progress or collapsed to illegal positions depending on the exact configuration. Interestingly, in Figure C5, the model collapse happens quite a lot faster than for DDPO in Figure C1 despite us using a lower learning rate $3 \cdot 10^{-6}$ with $\mu = 8$. We suspect the faster collapse could be related to ESPO making more efficient updates due to the sequence-level nature of the algorithm.

After observing model collapse, we enable all diversity filtering mechanisms described in Section 4.2.4. In Figure C6, we observe that the model collapses to generate illegal positions. Although, the model improved in Figure C5, we observe no meaningful improvement in Figure C6 even in the beginning of the training despite using the same training algorithm. At around 400 training steps, we observe huge gradient spikes and despite using gradient norm clipping, the model collapses to generate completely empty boards with no pieces. In addition to this run, we experimented with ESPO a lot, but were unable to get meaningful improvement to the model without complete mode collapse.



(a) Reward over training steps.



(b) Puzzle rate over training steps.

Figure C5: An illustration of ESPO training with just legality, uniqueness and counter-intuitiveness based reward. We observe model collapse at around 110 training steps.

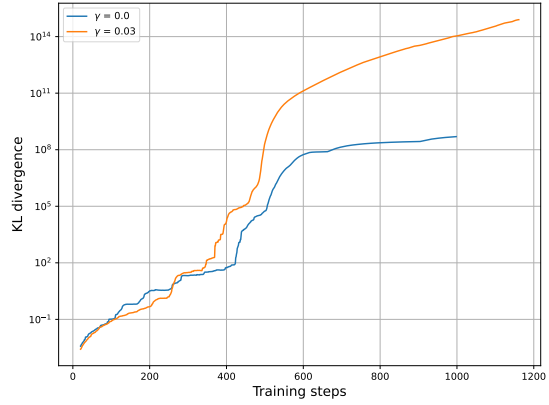
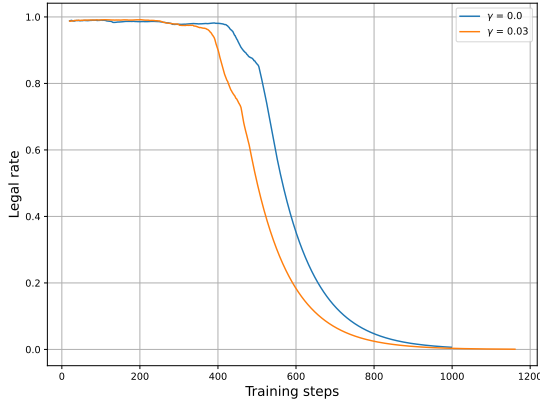


Figure C6: The reinforcement learning training using ESPO with all diversity filtering mechanisms enabled. We observe that the model collapses and starts to generate illegal positions.

D Implementation details

We provide an open-source implementation of the training algorithm we use at <https://github.com/naapeli/Generating-open-source-chess-puzzles>. All chess position evaluations were performed using the Stockfish engine [15]. To ensure reproducibility, we utilized a specific development build (dev-20260111-eb5a65ae), compiled on January 11, 2026, just before the release of Stockfish 18.